

Теория

Основы языка Go

1. Какие основные принципы и философия языка Go?

Go создан с фокусом на простоту, читаемость и продуктивность разработки. Ключевые принципы: - **Простота**: минималистичный синтаксис, отсутствие избыточных возможностей - **Явность**: код должен быть понятен без глубокого знания языка - **Компиляция**: статическая типизация и быстрая компиляция - **Конкурентность**: встроенная поддержка горутин и каналов - **Производительность**: близкая к C производительность при простоте использования - **Стандартизация**: единый стиль кода через `gofmt` и `golint` - **Одна задача - один способ**: избегание множественных способов решения одной задачи

2. В чем разница между `var`, `:=` и `new()` ?

- `var`: объявляет переменную с zero value типа. Можно использовать вне функций, позволяет указать тип явно: `var x int` или `var x = 10`
- `:=`: короткое объявление переменной с автоматическим выводом типа. Работает только внутри функций. Одновременно объявляет и инициализирует: `x := 10`
- `new()`: выделяет память в куче, инициализирует zero value и возвращает указатель на тип: `x := new(int)` эквивалентно `var x *int = new(int)`, где `*x == 0`

3. Что такое zero value в Go и для каких типов?

Zero value — значение по умолчанию для переменной, объявленной без инициализации: - Числовые типы: `0` (int, float, complex) - Булевы: `false` - Строки: `""` (пустая строка) - Указатели, слайсы, каналы, интерфейсы, функции: `nil` - Структуры: все поля имеют zero value - Массивы: все элементы имеют zero value

4. Как работает `defer` и в каком порядке выполняются `defer`-вызовы?

`defer` откладывает выполнение функции до момента возврата из текущей функции. Выполняется даже при панике или раннем `return`. Порядок выполнения: LIFO (Last In, First Out) — последний `defer` выполняется первым. Аргументы функции в `defer` вычисляются немедленно, но сама функция вызывается позже.

5. Что такое `init()` функция и когда она вызывается?

`init()` — специальная функция, автоматически вызываемая перед `main()`. В пакете может быть несколько `init()` функций — они выполняются в порядке объявления. Вызывается один раз при импорте пакета. Используется для инициализации пакета, регистрации компонентов, проверки зависимостей.

6. Как работает `recover()` и где его можно использовать?

`recover()` останавливает панику и возвращает значение, переданное в `panic()`. Работает только внутри `defer` функций. Если паники нет, возвращает `nil`. Используется для обработки критических ошибок, логирования паник, graceful degradation. Нельзя использовать для обычной обработки ошибок — только для восстановления после паники.

7. Что такое `iota` и как она работает?

`iota` — предопределенный идентификатор для создания последовательных констант. В каждом `const` блоке начинается с 0 и инкрементируется на 1 для каждой константы. Сбрасывается в каждом новом `const` блоке. Используется для enum-подобных констант, битовых масок, вычисляемых констант.

Структуры данных

8. Как устроены слайсы (slices) в Go? Что такое capacity и length?

Слайс — это дескриптор массива, состоящий из трех полей: указатель на базовый массив, длина (length) и емкость (capacity). Length — количество элементов в слайсе, capacity — размер базового массива. Слайс не хранит данные, а ссылается на часть массива. При создании через `make([]T, len, cap)` или литерал `[]T{...}` создается базовый массив.

9. Как работает функция `append()` и когда происходит переаллокация?

`append()` добавляет элементы в конец слайса. Если capacity достаточна, элементы добавляются в существующий массив. При нехватке места происходит переаллокация: создается новый массив большего размера (обычно в 2 раза), данные копируются, возвращается новый слайс. Старый массив остается в памяти до сборки мусора. Переаллокация происходит при `len(slice) == cap(slice)`.

10. Как устроена `map` в Go? Какая структура данных лежит в основе?

Map реализована как хеш-таблица. Внутри содержит массив бакетов (buckets), каждый бакет — связанный список пар ключ-значение. При добавлении элемента вычисляется хеш ключа, определяется бакет, элемент добавляется в список бакета. При переполнении (load factor > 6.5) происходит рехеширование — создается новая таблица в 2 раза больше, все элементы перераспределяются.

11. Как решаются коллизии в `map`?

Коллизии решаются методом цепочек (chaining). Каждый бакет содержит связанный список пар ключ-значение. При коллизии (одинаковый хеш) новый элемент добавляется в список того же бакета. Поиск проходит по списку бакета, сравнивая ключи напрямую после проверки хеша.

12. Какая сложность операций `get`, `put`, `delete` в `map`?

В среднем $O(1)$, в худшем случае $O(n)$ при вырождении всех элементов в один бакет (крайне редко). На практике операции близки к $O(1)$ благодаря равномерному распределению хешей и автоматическому рехешированию при переполнении.

13. Что такое `sync.Map` и когда его использовать вместо обычной `map`?

`sync.Map` — потокобезопасная реализация `map` из пакета `sync`. Используется когда: много горутин читают, но редко пишут; ключи стабильны во времени; нужна потокобезопасность без внешних мьютексов. Не подходит для частых записей — обычная `map` с `sync.RWMutex` будет эффективнее. `sync.Map` оптимизирована для read-heavy сценариев.

14. Почему нельзя использовать `map` как ключ в другой `map`?

Ключи `map` должны быть сравнимыми типами (comparable). `map` не является comparable типом, так как сравнение `map` требует сравнения всех элементов, что не определено в Go. Только базовые типы, указатели, структуры и массивы без `map/slice/func` полей могут быть ключами.

15. В чем разница между массивом и слайсом?

Массив — фиксированный размер, тип включает размер: `[5]int`. При передаче копируется целиком. Слайс — динамический, тип без размера: `[]int`. При передаче копируется только дескриптор (указатель, len, cap). Массивы используются редко, слайсы — основной способ работы с последовательностями.

16. Что произойдет при копировании слайса? Копируются ли элементы?

При присваивании `s2 := s1` копируется только дескриптор слайса (указатель, len, cap). Элементы не копируются — оба слайса ссылаются на один базовый массив. Изменения через один слайс видны через другой. Для копирования элементов используется `copy(dst, src)` или `append([]T{}, src...)`.

17. Что из себя представляют строки в Go? Как они устроены?

Строка в Go — это неизменяемая (immutable) последовательность байтов. Внутри строка представлена структурой из двух полей: указатель на массив байтов и длина (length). Строки хранятся в UTF-8 кодировке. При присваивании `s2 := s1` копируется только дескриптор (указатель + длина), данные не копируются. Строки нельзя изменять напрямую — любая операция создает новую строку. Для модификаций эффективнее использовать `strings.Builder` или `[]byte`. При итерации через `range` происходит декодирование UTF-8 в руны, при индексации `s[i]` — доступ к байтам.

Горутины и конкурентность

18. Что такое горутина и чем она отличается от системного потока?

Горутина — легковесный поток выполнения, управляемый runtime Go. Отличается от системного потока: меньший размер стека (2KB vs 1-2MB), быстрое создание/переключение, управление планировщиком Go, а не ОС. Множество горутин выполняется на меньшем количестве системных потоков (M:N модель). Переключение между горутинами дешевле, чем между потоками ОС.

19. Как работает планировщик горутин (Goroutine Scheduler) в Go?

Планировщик использует M:N модель: M горутин выполняются на N системных потоках ($M \gg N$). Компоненты: M (machine) — системные потоки, G (goroutine) — горутины, P (processor) — контексты процессоров. P содержит локальную очередь горутин. При блокировке горутины (syscall, channel) M передает P другому потоку. Work-stealing: простаивающий P может "украсть" горутины из другой очереди.

20. Что такое M:N модель и как она реализована в Go?

M:N модель — множество пользовательских потоков (горутин) выполняется на меньшем количестве системных потоков. Преимущества: быстрое переключение контекста, эффективное использование CPU, масштабируемость. В Go: планировщик управляет распределением горутин по системным потокам, автоматически балансирует нагрузку, обрабатывает блокирующие операции без блокировки всего потока.

21. Что такое context и для чего он используется?

context.Context — интерфейс для передачи сигналов отмены, таймаутов и значений между горутинами. Используется для: отмены долгих операций, установки дедлайнов, передачи request-scoped данных. Создается через context.Background(), context.TODO(), context.WithCancel(), context.WithTimeout(). Должен передаваться как первый параметр функции.

22. Как правильно отменять горутины через context?

Создать контекст с отменой: ctx, cancel := context.WithCancel(parent). Передать ctx в горутины. В горутине проверять ctx.Done() в цикле или использовать select с case <-ctx.Done(). Вызвать cancel() для отмены всех дочерних операций. Контекст распространяет отмену вниз по дереву вызовов.

23. Что такое context.WithTimeout и context.WithCancel?

context.WithTimeout(ctx, duration) создает контекст с автоматической отменой через указанное время. context.WithCancel(ctx) создает контекст с ручной отменой через cancel(). Оба возвращают дочерний контекст и функцию отмены. Отмена родительского контекста автоматически отменяет дочерние.

24. Что такое context.WithValue и когда его использовать?

context.WithValue(ctx, key, value) добавляет значение в контекст. Используется для передачи request-scoped данных (request ID, user info, trace ID). Ключи должны быть сравнимыми, рекомендуется использовать кастомные типы. Не использовать для передачи опциональных параметров — это антипаттерн. Только для данных, которые должны пройти через весь call chain.

25. Что такое sync.WaitGroup и как его использовать?

sync.WaitGroup — счетчик для ожидания завершения группы горутин. Методы: Add(delta) — увеличивает счетчик, Done() — уменьшает на 1 (вызывать в defer), Wait() — блокируется до обнуления счетчика. Используется для синхронизации: запустить N горутин, дождаться их завершения перед продолжением.

26. В чем разница между sync.Mutex и sync.RWMutex?

sync.Mutex — обычный мьютекс, один писатель или один читатель. sync.RWMutex — read-write мьютекс: множественные читатели (RLock()) или один писатель (Lock()). RWMutex эффективнее при read-heavy нагрузке, так как читатели не блокируют друг друга. Писатель блокирует всех читателей и других писателей.

27. Что такое sync.Once и когда его применять?

sync.Once гарантирует однократное выполнение функции через Do(f). Используется для: ленивой инициализации сингلتонов, одноразовой настройки, инициализации глобальных ресурсов. Потокобезопасен, эффективнее чем мьютекс для одноразовых операций.

28. Что такое atomic операции и когда их использовать?

atomic операции — атомарные операции над переменными из пакета sync/atomic. Гарантируют thread-safe доступ без мьютексов для простых операций (инкремент, сравнение-и-замена, загрузка/сохранение). Используются для: счетчиков, флагов, lock-free структур данных. Быстрее мьютексов для простых операций, но ограничены базовыми типами.

29. Что такое data race и как его обнаружить?

Data race — ситуация, когда несколько горутин одновременно обращаются к одной переменной, и хотя бы одна записывает. Может привести к неопределенному поведению. Обнаруживается: go run -race, go test -race, go build -race. Race detector отслеживает доступы к памяти и сообщает о потенциальных гонках. Замедляет выполнение в 2-20 раз.

30. Как работает -race флаг в Go?

Race detector инструментрует код для отслеживания доступа к памяти. Во время выполнения отслеживает: какие горутины обращаются к памяти, в каком порядке, есть ли синхронизация. При обнаружении потенциальной гонки выводит отчет с stack trace. Используется только для тестирования и разработки, не для production из-за накладных расходов.

31. Что такое goroutine leak и как его предотвратить?

Goroutine leak — ситуация, когда горутина продолжает работать и не завершается, потребляя память и ресурсы. Причины: ожидание на заблокированном канале, бесконечный цикл без условия выхода, забытые фоновые задачи. Предотвращение: использовать `context` для отмены, закрывать каналы после завершения работы, использовать таймауты, применять `select` с `case <- ctx.Done()`, тестировать с помощью детекторов утечек (например, `goleak`).

32. Что произойдет если main функция завершится раньше горутин?

Программа немедленно завершится вместе со всеми запущенными горутинами. Горутины не успеют завершить свою работу. Для ожидания завершения горутин используется: `sync.WaitGroup`, каналы для синхронизации, или `context` для управления жизненным циклом. Main функция должна явно ожидать завершения всех важных горутин.

33. Как ограничить количество одновременно работающих горутин?

Использовать семафор через буферизованный канал: `sem := make(chan struct{}, N)`. Перед запуском задачи `sem <- struct{}{}`, после завершения `<- sem`. Альтернативы: worker pool паттерн (N горутин-воркеров обрабатывают задачи из канала), пакеты golang.org/x/sync/semaphore или `errgroup.WithContext` с ограничением. Это предотвращает исчерпание ресурсов при массовом создании горутин.

34. Что такое GOMAXPROCS и как это влияет на производительность?

`GOMAXPROCS` задает максимальное количество системных потоков (M), которые могут одновременно выполнять Go код. По умолчанию равно количеству CPU ядер (`runtime.NumCPU()`). Изменяется через `runtime.GOMAXPROCS(n)` или переменную окружения `GOMAXPROCS`. Увеличение не всегда улучшает производительность — зависит от типа задачи (CPU-bound vs I/O-bound). Для I/O-bound задач значение может быть выше количества ядер.

35. Можно ли получить ID горутины и стоит ли это делать?

Технически можно через рефлексии или парсинг `stack trace`, но это антипаттерн. Go намеренно не предоставляет API для получения goroutine ID. Причины: поощряет правильные паттерны синхронизации через каналы и `context`, избирает thread-local storage антипаттернов, упрощает reasoning о коде. Если нужно отслеживать выполнение — используйте `context.WithValue` для передачи идентификаторов запросов.

36. Как безопасно передавать данные между горутинами?

Каналы — идиоматичный способ: `ch := make(chan Type)`, отправка `ch <- value`, получение `value := <- ch`. Для shared state используйте мьютексы (`sync.Mutex`, `sync.RWMutex`). Следуйте принципу "Don't communicate by sharing memory; share memory by communicating". Альтернативы: `sync.Map` для concurrent map, `atomic` для простых значений, immutable структуры (передача копий).

Каналы (Channels)

37. Как работают каналы в Go? Буферизованные vs небуферизованные

Канал — типобезопасная очередь для коммуникации между горутинами. Небуферизованный канал: отправка блокируется до получения, получение блокируется до отправки. Синхронная коммуникация. Буферизованный канал: имеет очередь фиксированного размера, отправка блокируется только при заполнении буфера, получение — при пустом буфере. Асинхронная коммуникация до заполнения буфера.

38. Что произойдет при записи в закрытый канал?

Вызовет панику `panic: send on closed channel`. Закрытый канал нельзя использовать для отправки. Перед отправкой нужно проверять, не закрыт ли канал, или использовать `recover()` для обработки паники. Обычно закрывает канал отправитель (producer), а получатели (consumers) проверяют второе значение `val, ok := <-ch`.

39. Что произойдет при чтении из закрытого канала?

Чтение из закрытого канала возвращает zero value типа и `false` как второе значение: `val, ok := <-ch`, где `ok == false` означает закрытие канала. Можно читать бесконечно — всегда будет zero value. Это позволяет получателям определить, что канал закрыт, и завершить работу.

40. Как правильно закрывать каналы?

Закрывать должен отправитель (producer), а не получатель. Закрывать только один раз — повторное закрытие вызывает панику. Использовать `defer close(ch)` для гарантии закрытия. Получатели проверяют `val, ok := <-ch` и завершаются при `ok == false`. Для множественных отправителей использовать дополнительный канал или `sync.Once`.

41. Что такое `select` и как он работает?

`select` — конструкция для ожидания нескольких каналов одновременно. Выполняет один из готовых `case`, выбирая случайно при нескольких готовых. Блокируется, если все каналы не готовы (если нет `default`). Используется для: мультиплексирования каналов, таймаутов, неблокирующих операций с `default`.

42. Что такое `default` в `select` и когда его использовать?

`default` в `select` выполняется немедленно, если ни один канал не готов. Делает операцию неблокирующей. Используется для: проверки готовности канала без блокировки, реализации таймеров, обработки приоритетных каналов. Без `default` `select` блокируется до готовности хотя бы одного канала.

43. Как использовать `select` для таймаутов?

Использовать `time.After(duration)` в одном из `case`: `select { case val := <-ch: ... case <-time.After(5*time.Second): ... }`. Или `context.WithTimeout` с `case <-ctx.Done()`. Таймаут срабатывает, если операция не завершилась за указанное время. `time.After` создает новый таймер каждый раз, для повторного использования лучше `time.NewTimer`.

44. Что такое `nil` канал и как он ведет себя в `select`?

`nil` канал никогда не готов — операции с ним всегда блокируются. В `select` `case` с `nil` каналом никогда не выберется. Используется для динамического включения/отключения каналов в `select`: установить канал в `nil` для отключения, восстановить для включения. Позволяет условно участвовать в `select`.

45. Как реализовать паттерн fan-in и fan-out с помощью каналов?

Fan-out: одна горутина читает из канала и распределяет работу между несколькими worker-горутинami. Fan-in: несколько горутин отправляют в один канал, одна горутина собирает результаты. Реализация: для fan-out запустить N workers, читающих из общего канала. Для fan-in использовать `select` для чтения из нескольких каналов в один, или закрыть входные каналы и собрать через `range`.

Интерфейсы

46. Как работают интерфейсы в Go? Что такое duck typing?

Интерфейс в Go — набор методов. Тип реализует интерфейс неявно, если содержит все методы интерфейса (structural typing, "duck typing"). Реализация проверяется во время компиляции. Интерфейсное значение содержит указатель на данные и таблицу методов. Duck typing: "если что-то ходит как утка и крикает как утка, то это утка" — тип соответствует интерфейсу по поведению, а не по объявлению.

47. Что такое пустой интерфейс `interface{}` и когда его использовать?

`interface{}` — интерфейс без методов, реализуется любым типом. Используется для: работы с неизвестными типами, generic-подобной функциональности (до Go 1.18), сериализации, рефлексии. Требует type assertion для использования. В Go 1.18+ предпочтительнее использовать `any` (алиас для `interface{}`) или дженерики.

48. В чем разница между `any` и `interface{}`?

`any` — это алиас для `interface{}`, введенный в Go 1.18. Функционально идентичны. `any` более читаемый и явно выражает намерение работать с любым типом. Рекомендуется использовать `any` в новом коде для лучшей читаемости. Оба компилируются одинаково.

49. Как работает type assertion и type switch?

Type assertion: `val, ok := x.(Type)` — проверяет, является ли `x` типом `Type`, возвращает значение и булев флаг.

`val := x.(Type)` — паникует при несоответствии. Type switch: `switch x.(type) { case int: ... case string: ... }` — переключается по типу значения. Используется для работы с `interface{}` / `any`, определения конкретного типа в runtime.

50. Что такое embedding интерфейсов?

Embedding — встраивание интерфейса в другой интерфейс. Все методы встроенного интерфейса становятся частью внешнего. Позволяет комбинировать интерфейсы: `type ReadWriter interface { Reader; Writer }`. Тип, реализующий `ReadWriter`, должен реализовать методы и `Reader`, и `Writer`. Упрощает композицию интерфейсов.

51. Можно ли реализовать интерфейс для типа из другого пакета?

Да, можно. Интерфейсы в Go используют структурную типизацию — тип автоматически реализует интерфейс, если содержит нужные методы, независимо от пакета. Это позволяет расширять функциональность чужих типов без модификации исходного кода. Например, можно сделать тип из стандартной библиотеки реализацией кастомного интерфейса.

52. Что такое `io.Reader` и `io.Writer`?

`io.Reader` — интерфейс для чтения: `Read([]byte) (int, error)`. `io.Writer` — интерфейс для записи: `Write([]byte) (int, error)`. Фундаментальные интерфейсы для работы с потоками данных. Реализованы файлами, сетью, буферами, строками. Позволяют писать универсальный код, работающий с любым источником/приемником данных. Основа многих абстракций в Go.

53. Как работает сборщик мусора (GC) в Go?

GC в Go — concurrent mark-and-sweep сборщик. Работает параллельно с программой (не останавливает выполнение полностью). Использует **триколорный алгоритм** маркировки: белый (не посещён), серый (посещён, потомки не проверены), чёрный (посещён со всеми потомками). Инвариант: нет прямых ссылок от чёрного к белому — это позволяет маркировать параллельно с программой. Фазы: mark (достижимые объекты), sweep (очистка недостижимых). Настраивается через `GOGC` (по умолчанию 100%). Цель: минимизировать паузы при сохранении низкого overhead.

Go 1.26 — Green Tea GC (по умолчанию). С Go 1.26 вместо старого маркера включён **Green Tea** (в Go 1.25 был экспериментом). Модель GC не меняется: тот же concurrent mark-and-sweep и триколорная абстракция, новых write barrier нет. Меняется только **как** проходит фаза mark: маркировка и сканирование **мелких объектов** идёт с учётом locality — объекты обходятся **по span'ам** (кускам кучи) последовательно, а не через случайный pointer chasing по всей памяти. Это даёт меньше cache miss'ов и лучше масштабируется на несколько ядер. На GC-нагруженных программах ожидается **~10–40% overhead** от сборки; на новых amd64 (Intel Ice Lake, AMD Zen 4+) ещё ~10% за счёт vector-инструкций при сканировании. Количество аллокаций Green Tea **не уменьшает** — только ускоряет сбор. Отключить: `GODEXPERIMENT=nogreenteagc` при сборке (opt-out планируют убрать в Go 1.27).

54. Что такое escape analysis и как он работает?

Escape analysis — анализ компилятора, определяющий, может ли переменная "убежать" из стека в кучу. Если переменная используется после возврата функции или передается в другую горутину, она размещается в куче. Цель: максимизировать размещение в стеке (быстрее). Можно проверить через `go build -gcflags=-m`. Влияет на производительность и GC pressure.

55. Когда переменная попадает в кучу (heap), а когда остается в стеке?

В кучу: если используется после возврата функции, передается в другую горутину, размер неизвестен на этапе компиляции, слишком большая для стека, адрес передается в функцию, которая может сохранить его. В стеке: локальные переменные, используемые только внутри функции, размер известен на этапе компиляции. Компилятор старается максимизировать размещение в стеке.

56. Что такое выравнивание (alignment) структур и почему это важно?

Выравнивание — размещение полей структуры по границам, кратным размеру типа (для эффективного доступа к памяти). Компилятор добавляет padding (пустые байты) между полями. Важно для: производительности (невыверенный доступ медленнее), размера структуры (порядок полей влияет на размер), совместимости с C структурами. Можно проверить через `unsafe.Offsetof`, `unsafe.Sizeof`.

57. Как оптимизировать использование памяти в Go?

Стратегии: минимизировать аллокации (переиспользование буферов, object pooling), уменьшить размер структур (оптимизация порядка полей), использовать слайсы с предварительной capacity, избегать утечек (закрывать каналы, отменять контексты), профилировать через `pprof`, использовать `sync.Pool` для временных объектов, минимизировать escape в кучу.

Ошибки и обработка ошибок

58. Как обрабатываются ошибки в Go? Почему нет исключений?

Ошибки в Go — обычные значения, возвращаемые как последний параметр функции. Явная обработка через проверку `if err != nil`. Нет исключений для: явности (видно все места обработки ошибок), производительности (нет overhead try/catch), простоты (нет сложных стеков вызовов). Паника (`panic`) используется только для критических ошибок, которые нельзя обработать. Обычные ошибки — через возвращаемое значение.

59. Что такое `error` интерфейс?

`error` — встроенный интерфейс: `type error interface { Error() string }`. Любой тип с методом `Error() string` является ошибкой. Стандартные реализации: `errors.New()`, `fmt.Errorf()`. Позволяет создавать кастомные типы ошибок с дополнительной информацией. Проверка через `err != nil`, сравнение через `errors.Is()`, `errors.As()`.

60. Как правильно оборачивать ошибки? `fmt.Errorf` VS `errors.Wrap`

`fmt.Errorf("описание: %w", err)` — оборачивает ошибку с контекстом, сохраняя исходную через `%w` (Go 1.13+). `errors.Wrap(err, "описание")` — из пакета `github.com/pkg/errors`, добавляет stack trace. Рекомендуется `fmt.Errorf` с `%w` для совместимости со стандартной библиотекой. Проверка обернутых ошибок через `errors.Is()`, `errors.As()`. Добавлять контекст на каждом уровне абстракции.

61. Что такое `errors.Is` и `errors.As`?

`errors.Is(err, target)` — проверяет, является ли ошибка или любая обернутая ошибка равной `target`. Работает с обернутыми ошибками через `%w`. `errors.As(err, target)` — проверяет, можно ли привести ошибку к типу `target`, и присваивает значение. Работает с обернутыми ошибками и `type assertion`. Оба используются для проверки обернутых ошибок вместо прямого сравнения.

62. Как создать кастомный тип ошибки?

Создать тип с методом `Error() string`: `type MyError struct { Code int; Msg string }` с `func (e MyError) Error() string { return e.Msg }`. Можно добавить методы для проверки: `func (e MyError) Is(target error) bool` для `errors.Is()`, `func (e MyError) As(target interface{}) bool` для `errors.As()`. Использовать для ошибок с дополнительной информацией (коды, контекст, recoverable состояния).

63. Как работают `panic` и `recover`? Чем `panic` отличается от `error`?

error — обычное значение, часть сигнатуры функции. Вызывающий **обязан** проверить `if err != nil`. Ошибка — ожидаемый сбой: сеть, валидация, «не найдено». Управление идёт явно, стек не раскручивается.

panic — механизм исключений runtime: раскрутка стека, выполнение всех `defer` по пути вверх. Останавливается только `recover()` **внутри defer** той же горутины. Если `recover` не поймал — процесс падает (кроме `panic` в горутине — умирает только она).

	<code>error</code>	<code>panic</code>
Когда	ожидаемые сбои	баг, нарушен инвариант, <code>init()</code>
Кто обрабатывает	каждый caller	<code>recover</code> выше по стеку
Горутина	текущая продолжает	раскрутка до <code>recover</code> или crash
В библиотеках	всегда	почти никогда (кроме <code>Must*</code>)

panic в горутине: не убивает весь процесс, но запрос/воркер обрывается. В HTTP — middleware с `defer recover()` и лог stack trace.

recover: возвращает аргумент `panic(x)`. Вне `defer` — бесполезен. Паттерн:

```
defer func() {
    if r := recover(); r != nil {
        log.Printf("panic: %v\n%s", r, debug.Stack())
    }
}()
}
```

На собесе: «ошибку файла — `error`, nil pointer в проде — баг, но в handler ловим `recover`». Не использовать `panic` для control flow.

Тестирование

64. Как писать unit-тесты в Go?

Тесты — функции вида `func TestXxx(t *testing.T)` в файлах `*_test.go`. Запуск: `go test`. Использовать `t.Error()`, `t.Fatal()` для ошибок. `t.Fatal()` останавливает тест, `t.Error()` продолжает. Имя теста должно начинаться с `Test`. Можно использовать подтесты через `t.Run()`. Тесты выполняются параллельно по умолчанию (если не использовать флаги).

65. Что такое table-driven tests?

Table-driven tests — паттерн тестирования с таблицей тест-кейсов. Массив структур с входными данными и ожидаемым результатом, цикл по таблице с `t.Run()` для каждого кейса. Преимущества: компактность, легко добавлять кейсы, единообразная структура. Стандартный подход в Go для тестирования функций с множественными входными данными.

66. Как использовать `t.Helper()`?

`t.Helper()` помечает функцию как test helper. Stack trace при ошибке будет указывать на вызывающий код, а не на helper. Используется в вспомогательных функциях тестирования для правильного отображения места ошибки. Вызывать в начале helper-функции. Улучшает читаемость отчетов об ошибках.

67. Что такое `testify` и для чего он нужен?

`testify` — библиотека расширений для тестирования. Компоненты: `assert` (проверки с понятными сообщениями), `require` (проверки с остановкой теста), `suite` (организация тестов в наборы), `mock` (мокирование). Упрощает написание тестов, улучшает читаемость. Не обязательна, но популярна в сообществе Go.

68. Как писать benchmark тесты?

Benchmark — функции вида `func BenchmarkXxx(b *testing.B)`. Запуск: `go test -bench=.` Использовать `b.N` для количества итераций. Запускать несколько раз для стабильности: `-benchtime`, `-count`. Избегать побочных эффектов между итерациями. Использовать `b.ResetTimer()`, `b.StopTimer()` для исключения setup из измерений. Сравнивать через `benchstat` или `benchcmp`.

69. Как измерять покрытие кода тестами?

Запуск с флагом: `go test -cover`. Детальный отчет: `go test -coverprofile=coverage.out`, затем `go tool cover -html=coverage.out`. Показывает процент покрытых строк. Цель: высокое покрытие критических путей, не обязательно 100%. Использовать для выявления непротестированных участков кода. Интегрировать в CI/CD.

70. Что такое `httptest` и как тестировать HTTP handlers?

`httptest` — пакет для тестирования HTTP кода. `httptest.NewRequest()` создает `*http.Request`, `httptest.NewRecorder()` создает `http.ResponseWriter` для записи ответа. Тестировать handlers изолированно без реального сервера. Проверять статус код, заголовки, тело ответа. Для интеграционных тестов использовать `httptest.NewServer()` для создания тестового HTTP сервера.

Пакеты и модули

71. Как работает система модулей в Go (go.mod)?

Модули — единица зависимостей с версионированием. `go.mod` определяет модуль: путь модуля, версия Go, зависимости с версиями. Управление: `go mod init`, `go mod tidy` (синхронизация), `go mod download`, `go get` (добавление зависимостей). Версионирование через семантические версии (v1.2.3) или псевдо-версии для коммитов. Заменяет устаревший `GOPATH`. Опционально `go mod vendor` создаёт `vendor/` с копиями зависимостей для офлайн-сборки (`go build -mod=vendor`).

72. Как работает `go.sum` файл?

`go.sum` содержит криптографические хеши зависимостей для проверки целостности. Генерируется автоматически при `go mod download`, `go build`. Содержит хеши для всех версий зависимостей (прямых и транзитивных). Используется для: проверки неизменности зависимостей, обнаружения подмены, воспроизводимых сборок. Должен коммититься в VCS вместе с `go.mod`.

73. Что такое `internal` пакет?

`internal/` — специальная директория, пакеты внутри доступны только родительскому модулю и его подмодулям. Защищает внутренние API от внешнего использования. Пакет `a/b/internal/c` доступен только пакетам внутри `a/b/`, но не `a/d/`. Используется для: скрытия реализации, предотвращения зависимости от внутренних API, организации приватных компонентов модуля.

74. Как спроектировать высоконагруженную систему на Go?

Принципы: горизонтальное масштабирование (stateless сервисы), асинхронная обработка (каналы, очереди), кэширование (in-memory, Redis), connection pooling, graceful degradation, circuit breaker, rate limiting, мониторинг и метрики. Использовать горутины для конкурентной обработки, избегать блокирующих операций, оптимизировать аллокации памяти, использовать профилирование для узких мест.

75. Какие паттерны проектирования часто используются в Go?

Популярные паттерны: Dependency Injection (через интерфейсы), Factory, Builder, Strategy (через интерфейсы), Observer (через каналы), Pipeline (через каналы), Worker Pool, Context для cancellation/timeout. Go предпочитает композицию наследованию, интерфейсы для абстракций, каналы для коммуникации. Многие классические паттерны упрощаются благодаря горутинам и каналам.

76. Как обеспечить отказоустойчивость микросервисов?

Стратегии: health checks, circuit breaker (предотвращение каскадных отказов), retry с exponential backoff, timeout на всех внешних вызовах, fallback механизмы, graceful degradation, мониторинг и алертинг, изоляция отказов (bulkhead pattern), rate limiting для защиты от перегрузки. Использовать `context` для таймаутов, библиотеки типа `hystrix-go` или `gobreaker` для circuit breaker.

77. Как реализовать circuit breaker на Go?

Circuit breaker предотвращает каскадные отказы, блокируя вызовы при превышении порога ошибок. Состояния: Closed (норма), Open (блокировка), Half-Open (тестирование восстановления). Реализация: счетчик ошибок, таймер для перехода в Half-Open, логика переключения состояний. Библиотеки: `sony/gobreaker`, `afex/hystrix-go`. Можно реализовать на основе счетчиков и мьютексов с проверкой состояния перед вызовом.

78. Как работать с SQL базами данных в Go?

Использовать пакет `database/sql` — стандартный интерфейс для работы с SQL БД. Драйверы: `pq` (PostgreSQL), `go-sql-driver/mysql`, `mattn/go-sqlite3`. Основные операции: `db.Open()`, `db.Query()`, `db.Exec()`, `db.Prepare()`. Всегда закрывать `rows` и `stmt`, обрабатывать ошибки, использовать `prepared statements` для безопасности и производительности. `Connection pooling` встроен в `database/sql`.

79. Что такое `database/sql` и как он работает?

`database/sql` — стандартный интерфейс для работы с SQL БД, абстракция над драйверами. Управляет `connection pool` автоматически: создает, переиспользует, закрывает соединения. Настройки: `SetMaxOpenConns()`, `SetMaxIdleConns()`, `SetConnMaxLifetime()`. Предоставляет типизированные методы: `Query()` для `SELECT`, `Exec()` для `INSERT/UPDATE/DELETE`, `Prepare()` для `prepared statements`. Абстрагирует различия между БД.

80. Как работают `connection pools` в Go?

`Connection pool` управляется `database/sql` автоматически. Настройки: `MaxOpenConns` (максимум открытых), `MaxIdleConns` (максимум простаивающих), `ConnMaxLifetime` (время жизни соединения). При запросе: берется соединение из пула, используется, возвращается. При нехватке создается новое (до `MaxOpenConns`). Простаивающие соединения закрываются после `ConnMaxLifetime`. Важно настроить под нагрузку для оптимальной производительности.

81. Как использовать транзакции?

Транзакции через `db.Begin()`, возвращает `*sql.Tx`. Методы: `tx.Exec()`, `tx.Query()`, `tx.Commit()`, `tx.Rollback()`. Всегда использовать `defer tx.Rollback()` для отката при ошибке, вызывать `tx.Commit()` при успехе. Транзакции изолируют операции, обеспечивают ACID свойства. Использовать для атомарных операций, состоящих из нескольких запросов. Не забывать закрывать транзакции.

82. Какие ORM используются в Go? (GORM, `sqlx` и т.д.)

Популярные библиотеки: GORM (полнофункциональный ORM с миграциями, ассоциациями), `sqlx` (расширение `database/sql` с удобной работой со структурами), `ent` (Facebook, code generation), `sqlboiler` (code generation). GORM — самый популярный, но тяжелее. `sqlx` — легковесный, ближе к SQL. Выбор зависит от сложности проекта: простые запросы — `sqlx`, сложные модели — GORM или `ent`.

83. Как работает `net/http` пакет?

`net/http` — стандартный пакет для HTTP клиентов и серверов. Сервер: `http.ListenAndServe()`, `http.HandleFunc()`, `http.Handler` интерфейс. Клиент: `http.Get()`, `http.Post()`, `http.Client` для настройки. Обработывает HTTP протокол, управляет соединениями, парсит заголовки. `http.Request` содержит метод, URL, заголовки, тело. `http.ResponseWriter` для записи ответа. Основа веб-разработки в Go.

84. В чем разница между `http.Handle` и `http.HandleFunc`?

`http.Handle(pattern, handler)` принимает `http.Handler` интерфейс (метод `ServeHTTP`). `http.HandleFunc(pattern, handler)` принимает функцию `func(http.ResponseWriter, *http.Request)`. `HandleFunc` — удобная обертка, конвертирующая функцию в `http.Handler`. Функционально эквивалентны, выбор зависит от стиля: функции — `HandleFunc`, структуры с состоянием — `Handle` с кастомным `Handler`.

85. Как создать middleware для HTTP handlers?

Middleware — функция, оборачивающая handler для добавления функциональности (логирование, аутентификация, CORS). Паттерн: `func(http.HandlerFunc) http.HandlerFunc` или `func(http.Handler) http.Handler`. Цепочка middleware через композицию функций. Примеры: логирование запросов, проверка авторизации, добавление заголовков, rate limiting. Можно использовать библиотеки типа `gorilla/mux` для удобной композиции.

86. Как работает `context` в HTTP запросах?

Каждый HTTP запрос имеет `context`, доступный через `r.Context()`. Автоматически отменяется при закрытии соединения клиентом. Используется для: передачи request-scoped данных (request ID, user info), таймаутов, отмены долгих операций. Передавать `ctx` во все асинхронные операции. `http.Request.WithContext()` создает новый запрос с контекстом. Критично для корректной обработки отмены запросов.

87. Как реализовать graceful shutdown HTTP сервера?

Использовать `http.Server.Shutdown(ctx)` вместо прямого закрытия. Процесс: перехватить сигнал (SIGTERM, SIGINT), вызвать `server.Shutdown()` с таймаутом, сервер перестанет принимать новые запросы и дожидается завершения текущих (до таймаута). После таймаута принудительно закрывает соединения. Позволяет обработать in-flight запросы, критично для production без потери данных.

Производительность и профилирование

88. Какие инструменты для профилирования есть в Go?

Основные инструменты: `pprof` (CPU, память, горютины, блокировки), `go tool trace` (детальная трассировка выполнения), `go test -bench` (бенчмарки), `go test -cpuprofile`, `go test -memprofile`. Встроенные в `runtime`, не требуют внешних инструментов. Интеграция через `net/http/pprof` для live профилирования работающих приложений. Стандартные инструменты покрывают большинство случаев.

89. Как использовать `pprof` и профилировать CPU/память?

Импортировать `"net/http/pprof"` и запустить HTTP сервер на `admin`-порту. Эндпоинты: `/debug/pprof/` (список), `/debug/pprof/profile` (CPU), `/debug/pprof/heap` (память). Анализ: `go tool pprof http://localhost:6060/debug/pprof/profile?seconds=30` или `go test -cpuprofile=cpu.prof`. Команды: `top`, `list` `FunctionName`, `web`. CPU показывает `hot paths`, `heap` — аллокации и удерживаемую память. В тестах: `go test -memprofile=mem.prof`.

90. Что такое `go tool trace`?

`go tool trace` — инструмент для детальной трассировки выполнения программы. Показывает: планирование горютин, GC события, системные вызовы, блокировки, сетевые операции. Генерация: `runtime/trace` пакет, `go test -trace=trace.out`. Визуализация: `go tool trace trace.out`, открывает веб-интерфейс. Полезен для анализа конкурентности, понимания поведения планировщика, диагностики проблем производительности.

91. Как оптимизировать производительность Go приложений?

Стратегии: профилирование для выявления узких мест (не угадывать), минимизация аллокаций (переиспользование, `pooling`), оптимизация `hot paths`, использование `sync.Pool` для временных объектов, избегание `escape` в кучу, оптимизация структур (выравнивание), использование `atomic` вместо мьютексов где возможно, `connection pooling` для БД, кэширование, `batch` обработка. Измерять до и после оптимизаций.

92. Какие уязвимости безопасности характерны для Go приложений?

Типичные уязвимости: SQL инъекции (неправильное использование строковых запросов), XSS (неэкранированный вывод), CSRF (отсутствие токенов), утечки секретов (в коде, логах), небезопасная десериализация, path traversal, недостаточная валидация входных данных, использование `unsafe` пакета, race conditions. Go защищает от некоторых уязвимостей (buffer overflows благодаря проверке границ), но требует внимательности разработчика.

93. Как защититься от SQL инъекций?

Всегда использовать prepared statements (`db.Prepare()`, `stmt.Exec()`), никогда не конкатенировать пользовательский ввод в SQL запросы. Параметризованные запросы: `db.Exec("SELECT * FROM users WHERE id = ?", userID)`. Библиотеки типа GORM, sqlx автоматически используют prepared statements. Валидировать и санитизировать входные данные. Использовать whitelist для допустимых значений где возможно.

94. Как безопасно работать с секретами?

Никогда не хранить секреты в коде или VCS. Использовать: переменные окружения, секретные менеджеры (HashiCorp Vault, AWS Secrets Manager), конфигурационные файлы вне VCS, Kubernetes secrets. В production использовать сервисы управления секретами. Не логировать секреты. Ротация секретов. Использовать минимальные права доступа (principle of least privilege). Шифрование секретов в покое и при передаче.

95. Что такое `unsafe` пакет и когда его использовать?

`unsafe` — пакет для низкоуровневых операций, обходящих систему типов Go. Функции: `unsafe.Pointer` (преобразование указателей), `unsafe.Sizeof`, `unsafe.Offsetof`, `unsafe.Alignof`. Использовать только когда: критична производительность, работа с C кодом, оптимизация структур данных, низкоуровневые системные вызовы. Опасен: может привести к панике, нарушению безопасности памяти, несовместимости между версиями Go. Избегать без крайней необходимости.

Сети и сетевые протоколы

96. Что такое модель TCP/IP?

TCP/IP — практическая модель сетевого взаимодействия из 4 уровней: Link (канальный, Ethernet), Internet (сетевой, IP/ICMP), Transport (транспортный, TCP/UDP), Application (прикладной, HTTP, DNS). Фактическая основа Интернета. Для backend достаточно понимать: IP маршрутизирует пакеты, TCP/UDP доставляют данные между процессами, HTTP — протокол приложения поверх TCP.

97. В чем разница между TCP и UDP?

TCP (Transmission Control Protocol) — надежный, с установлением соединения, гарантирует доставку и порядок пакетов, контроль перегрузки, медленнее. UDP (User Datagram Protocol) — ненадежный, без соединения, не гарантирует доставку и порядок, быстрее, меньше overhead. TCP для критичных данных (HTTP, FTP, email), UDP для потокового видео, игр, DNS где важна скорость, а потери допустимы.

98. Как работает TCP handshake (трехстороннее рукопожатие)?

Трехэтапное установление соединения:

- 1) SYN — клиент отправляет SYN пакет с начальным sequence number
 - 2) SYN-ACK — сервер отвечает SYN-ACK с подтверждением и своим sequence number
 - 3) ACK — клиент отправляет ACK, подтверждая получение. После этого соединение установлено, начинается передача данных.
- Завершение через 4-way handshake: FIN, ACK, FIN, ACK.

99. Что такое HTTP и какие у него методы?

HTTP (HyperText Transfer Protocol) — протокол прикладного уровня для передачи данных в веб. Основные методы: GET (получение ресурса), POST (создание ресурса), PUT (полное обновление), PATCH (частичное обновление), DELETE (удаление), HEAD (получение заголовков без тела), OPTIONS (допустимые методы для ресурса). Stateless протокол, работает поверх TCP, порт 80 (HTTP) или 443 (HTTPS).

100. В чем разница между HTTP/1.1, HTTP/2 и HTTP/3?

HTTP/1.1 — текстовый протокол, одно соединение на запрос (или keep-alive), head-of-line blocking. HTTP/2 — бинарный, мультиплексирование (несколько запросов в одном соединении), Server Push, компрессия заголовков (HPACK), приоритизация, но HOL blocking на уровне TCP. HTTP/3 — работает поверх QUIC (UDP), устраняет TCP HOL blocking, быстрое установление соединения, лучше на нестабильных сетях.

101. Что такое HTTPS и как работает SSL/TLS?

HTTPS — HTTP поверх SSL/TLS, обеспечивает шифрование, аутентификацию и целостность данных. TLS Handshake:

- 1) Client Hello — клиент отправляет поддерживаемые cipher suites
- 2) Server Hello — сервер выбирает suite, отправляет сертификат
- 3) клиент проверяет сертификат, генерирует session key, шифрует его публичным ключом сервера
- 4) сервер расшифровывает session key приватным ключом
- 5) начинается зашифрованная передача симметричным ключом.

102. Что такое DNS и как работает разрешение имен?

DNS (Domain Name System) — система преобразования доменных имен в IP-адреса. Процесс:

- 1) браузер проверяет кеш
- 2) запрос к локальному DNS resolver (ISP)
- 3) если нет в кеше — рекурсивный запрос: root server → TLD server (.com) → authoritative server домена
- 4) ответ возвращается и кешируется. DNS использует UDP (порт 53) для запросов, TCP для зонных трансферов. TTL определяет время жизни записи в кеше.

103. Что такое REST API? Какие у него принципы?

REST (Representational State Transfer) — архитектурный стиль для веб-сервисов. Принципы:

- 1) Stateless — каждый запрос независим, содержит всю необходимую информацию
- 2) Client-Server — разделение интересов
- 3) Cacheable — ответы должны определять, кешируемы ли они
- 4) Uniform Interface — единообразный интерфейс (HTTP методы, URI)
- 5) Layered System — клиент не знает о промежуточных слоях
- 6) Code on Demand (опционально) — сервер может передавать исполняемый код.

104. Что такое WebSocket и чем отличается от HTTP?

WebSocket — протокол полнодуплексной связи поверх TCP, позволяет двустороннюю передачу данных в реальном времени. Отличия от HTTP: постоянное соединение vs запрос-ответ, двунаправленная передача vs однонаправленная, меньше overhead для частых сообщений. Использование: чаты, онлайн игры, live updates. Устанавливается через HTTP Upgrade запрос, затем переключается на WebSocket протокол (порт 80/443).

105. Что такое порты и для чего они нужны?

Порты — числовые идентификаторы (0-65535) для различения приложений/сервисов на одном хосте. Диапазоны: 0-1023 (well-known, системные: HTTP 80, HTTPS 443, SSH 22, DNS 53), 1024-49151 (registered, пользовательские приложения), 49152-65535 (dynamic/private, временные). Порт + IP адрес = socket, уникально идентифицирует соединение. Протокол транспортного уровня (TCP/UDP) использует порты для мультиплексирования.

106. Что такое NAT и для чего он используется?

NAT (Network Address Translation) — технология преобразования IP-адресов, позволяет множеству устройств в частной сети использовать один публичный IP. Типы: Static NAT (1:1), Dynamic NAT (пул публичных IP), PAT/NAPT (Port Address Translation, множество частных на один публичный через разные порты). Решает проблему нехватки IPv4 адресов, обеспечивает базовую безопасность (скрывает внутреннюю структуру сети).

107. Что такое load balancer и какие типы бывают?

Load Balancer — компонент, распределяющий входящий трафик между несколькими серверами для повышения доступности и производительности. Уровни: L4 (Transport) — балансировка по IP/port, быстрая, не смотрит содержимое; L7 (Application) — балансировка по HTTP заголовкам, URL, cookies, медленнее, но гибче. Алгоритмы: Round Robin, Least Connections, IP Hash, Weighted. Примеры: nginx, HAProxy, AWS ELB/ALB, Envoy.

108. Что такое процесс и поток? В чем разница?

Процесс — независимая единица выполнения с собственным адресным пространством, ресурсами (память, дескрипторы файлов), контекстом выполнения. Поток (thread) — легковесная единица выполнения внутри процесса, разделяет адресное пространство и ресурсы процесса, имеет собственный стек и регистры. Различия: процессы изолированы, создание дороже, IPC сложнее; потоки разделяют память, создание дешевле, коммуникация проще, но нужна синхронизация.

109. Как процессы взаимодействуют между собой (IPC)?

IPC (Inter-Process Communication) — механизмы взаимодействия процессов:

- 1) Pipes (каналы) — однонаправленная передача данных между родственными процессами
- 2) Named Pipes (FIFO) — именованные каналы для несвязанных процессов
- 3) Message Queues — очереди сообщений
- 4) Shared Memory — общая память, самый быстрый, требует синхронизации
- 5) Sockets — сетевое взаимодействие локально или удаленно
- 6) Signals — асинхронные уведомления
- 7) Semaphores — синхронизация доступа к ресурсам.

110. Что такое системный вызов (syscall)?

Системный вызов — интерфейс между пользовательским процессом и ядром ОС для запроса привилегированных операций.

Процесс: приложение вызывает библиотечную функцию → переключение контекста из user mode в kernel mode → выполнение в ядре → возврат результата → переключение обратно в user mode. Примеры: `open()`, `read()`, `write()`. Syscall медленнее обычных вызовов из-за переключения контекста и проверок безопасности.

111. Что такое виртуальная память и зачем она нужна?

Виртуальная память — абстракция, позволяющая процессам использовать больше памяти, чем физически доступно, и изолировать адресные пространства. MMU (Memory Management Unit) транслирует виртуальные адреса в физические через таблицы страниц.

Преимущества: изоляция процессов (безопасность), простое управление памятью, memory overcommitment, поддержка swap. Каждый процесс видит свое адресное пространство начиная с 0, независимо от других.

112. Что такое page fault и когда он возникает?

Page fault — исключение, возникающее при обращении к странице памяти, которой нет в физической памяти. Типы:

- 1) Minor (soft) — страница в памяти, но не в таблице страниц процесса
- 2) Major (hard) — страница на диске (swap), требует I/O
- 3) Invalid — обращение к несуществующему адресу (Segmentation Fault). При page fault ядро загружает страницу в память, обновляет таблицу страниц, возобновляет выполнение. Major page faults медленные из-за дисковых операций.

113. Что такое context switching и почему он дорогой?

Context switching — переключение CPU с одного процесса/потока на другой. Операции: сохранение регистров, program counter, stack pointer текущего процесса; загрузка состояния следующего процесса; переключение таблиц страниц (для процессов). Дорогой из-за: сохранения/восстановления состояния, инвалидации TLB и кешей процессора, переключения в kernel mode. Частое переключение снижает производительность. Потоки дешевле процессов (не переключают адресное пространство).

114. Что такое файловый дескриптор?

Файловый дескриптор (file descriptor) — целое число, идентифицирующее открытый файл или I/O ресурс в процессе. Ядро поддерживает таблицу дескрипторов для каждого процесса. Стандартные: 0 (stdin), 1 (stdout), 2 (stderr). Операции: `open()` создает, `read()/write()` используют, `close()` освобождает. Ограничения: системный лимит (`ulimit -n`), процессный лимит. Дескриптор — абстракция над файлами, сокетами, pipes, устройствами.

115. Что такое signals в Unix/Linux?

Signals — асинхронные уведомления процессу о событиях. Типы: SIGINT (Ctrl+C, прерывание), SIGTERM (вежливое завершение), SIGKILL (принудительное завершение, нельзя обработать), SIGHUP (hangup, перезагрузка конфига), SIGSEGV (segmentation fault), SIGCHLD (дочерний процесс завершился). Обработка: default action, ignore, custom handler (через `signal()` или `sigaction()`). Сигналы могут прервать системные вызовы. `kill()` отправляет сигнал процессу. **SIGTERM** — основной сигнал для graceful shutdown в k8s.

116. Что такое Copy-on-Write (COW)?

Copy-on-Write — оптимизация для копирования данных: вместо немедленного копирования создаются ссылки на оригинал, реальное копирование происходит только при модификации. Используется в `fork()`, файловых системах (btrfs, ZFS), strings в некоторых языках. Преимущества: экономия памяти, быстрое копирование, ленивые вычисления. При записи происходит page fault и копирование страницы.

Базы данных (общие вопросы)

117. Что такое ACID и для чего нужны эти свойства?

ACID — набор свойств транзакций в БД:

- 1) Atomicity (атомарность) — транзакция выполняется полностью или не выполняется вообще
- 2) Consistency (согласованность) — транзакция переводит БД из одного корректного состояния в другое
- 3) Isolation (изолированность) — параллельные транзакции не влияют друг на друга
- 4) Durability (долговечность) — зафиксированные изменения сохраняются даже при сбое. Гарантирует надежность и корректность данных в многопользовательской среде.

118. Что такое транзакция и зачем она нужна?

Транзакция — последовательность операций с БД, выполняемая как единое целое. Начинается BEGIN/START TRANSACTION, завершается COMMIT (применение изменений) или ROLLBACK (отмена). Используется для: обеспечения целостности данных при множественных связанных операциях (перевод денег: списание + зачисление), изоляции параллельных операций, возможности отката при ошибках. Транзакции гарантируют ACID свойства.

119. Какие уровни изоляции транзакций существуют?

Четыре уровня (от слабого к сильному):

- 1) Read Uncommitted — видны незафиксированные изменения (dirty read)
- 2) Read Committed — видны только зафиксированные данные, но возможны non-repeatable read
- 3) Repeatable Read — повторное чтение возвращает те же данные, но возможны phantom read
- 4) Serializable — полная изоляция, как последовательное выполнение, самый медленный. Проблемы: Dirty Read (чтение незафиксированных), Non-Repeatable Read (данные изменились между чтениями), Phantom Read (новые строки появились/исчезли).

120. Что такое индекс и для чего он нужен?

Индекс — структура данных, ускоряющая поиск по таблице. Хранит отсортированные значения столбцов со ссылками на строки. Преимущества: быстрый поиск $O(\log n)$ вместо $O(n)$, ускоряет WHERE, JOIN, ORDER BY, GROUP BY. Недостатки: занимает место, замедляет INSERT/UPDATE/DELETE (нужно обновлять индекс), требует обслуживания. Типы: B-tree (сбалансированное дерево, универсальный), Hash (точное равенство), Bitmap (низкая кардинальность), Full-text (текстовый поиск).

121. Какие типы индексов существуют?

Основные типы:

- 1) B-tree — сбалансированное дерево, универсальный, поддерживает $>$, $<$, $=$, BETWEEN, ORDER BY
- 2) Hash — быстрый точный поиск ($=$), не поддерживает диапазоны
- 3) GiST/GIN (PostgreSQL) — для полнотекстового поиска, массивов, JSON
- 4) Bitmap — для низкой кардинальности (пол, статус), комбинирует несколько индексов
- 5) Clustered — данные физически отсортированы по индексу (Primary Key)
- 6) Non-clustered — отдельная структура. По составу: Single-column, Composite (многостолбцовый), Covering (включает все нужные столбцы).

122. Что такое нормализация БД и какие формы существуют?

Нормализация — процесс организации данных для уменьшения избыточности и аномалий. Нормальные формы:

- 1NF — атомарные значения, нет повторяющихся групп
- 2NF — выполнена 1NF, нет частичной зависимости от ключа
- 3NF — выполнена 2NF, нет транзитивной зависимости
- BCNF — каждый детерминант — суперключ
- 4NF, 5NF — более строгие.

Обычно достаточно 3NF. Денормализация — намеренное добавление избыточности для производительности (кеширование агрегатов, дублирование данных).

123. Что такое JOIN и какие типы бывают?

JOIN — операция объединения строк из двух или более таблиц по условию. Типы:

- 1) INNER JOIN — возвращает только совпадающие строки из обеих таблиц
- 2) LEFT JOIN (LEFT OUTER) — все строки из левой таблицы + совпадения из правой (NULL если нет)
- 3) RIGHT JOIN — все из правой + совпадения из левой
- 4) FULL OUTER JOIN — все строки из обеих таблиц
- 5) CROSS JOIN — декартово произведение, каждая строка с каждой
- 6) SELF JOIN — таблица соединяется сама с собой.

124. Что такое первичный ключ (Primary Key) и внешний ключ (Foreign Key)?

Primary Key — уникальный идентификатор строки в таблице, не может быть NULL, только один на таблицу. Обеспечивает уникальность и идентификацию записей, автоматически создает индекс. Foreign Key — столбец, ссылающийся на Primary Key другой таблицы, обеспечивает ссылочную целостность. Ограничивает: невозможно вставить несуществующую ссылку, удалить связанные записи (или CASCADE DELETE). Создает связи между таблицами (один-ко-многим, многие-ко-многим через junction table).

125. Что такое репликация и какие типы бывают?

Репликация — копирование данных с одного сервера (master/primary) на другие (slaves/replicas). Типы:

- 1) Синхронная — транзакция подтверждается после записи на реплики, надежнее, медленнее
- 2) Асинхронная — запись сначала на master, затем на реплики, быстрее, риск потери данных. По топологии: Master-Slave (один пишет, многие читают), Master-Master (все пишут и читают), Multi-master. Цели: масштабирование чтения, отказоустойчивость, географическое распределение, резервное копирование.

126. Что такое шардирование (sharding)?

Шардирование — горизонтальное разделение данных между несколькими БД/серверами. Каждый шард содержит подмножество данных по sharding key (например, $user_id \% N$). Преимущества: масштабирование записи и чтения, распределение нагрузки. Недостатки: сложность JOIN между шардами, транзакции через шарды сложны, ребалансировка данных. Стратегии: Range-based (диапазоны ключей), Hash-based (хеш функция), Geographic (по географии), Directory-based (таблица маппинга).

127. В чем разница между SQL и NoSQL базами данных?

SQL (реляционные) — структурированные данные, схема, ACID, мощные JOIN, вертикальное масштабирование, строгая типизация. Примеры: PostgreSQL, MySQL, Oracle. NoSQL — гибкая схема, масштабирование горизонтальное, eventual consistency, денормализация. Типы: Document (MongoDB, CouchDB), Key-Value (Redis, DynamoDB), Column-family (Cassandra, HBase), Graph (Neo4j). Выбор: SQL для сложных связей и транзакций, NoSQL для масштабирования и гибкости схемы.

128. Что такое CAP теорема?

CAP теорема — в распределенной системе невозможно одновременно гарантировать все три свойства:

- 1) Consistency (согласованность) — все узлы видят одинаковые данные
- 2) Availability (доступность) — каждый запрос получает ответ (успех или ошибку)
- 3) Partition Tolerance (устойчивость к разделению) — система работает при сетевых разрывах. Нужно выбрать два из трех. На практике: CP (ACID БД), AP (eventual consistency NoSQL). При network partition выбор между C и A.

129. Что такое денормализация и когда ее применять?

Денормализация — намеренное добавление избыточности в нормализованную БД для улучшения производительности. Методы: дублирование данных, хранение агрегатов (сумм, счетчиков), материализованные представления, кеширование вычисляемых полей. Применять когда: читаемость критична (чтение >> записи), сложные JOIN замедляют запросы, агрегации выполняются часто. Компромисс: быстрее чтение, медленнее запись, риск рассинхронизации, сложнее поддержка целостности.

130. Что такое explain plan и для чего используется?

EXPLAIN (или EXPLAIN ANALYZE) — инструмент для анализа плана выполнения запроса. Показывает: какие индексы используются, порядок JOIN, типы сканирования (Sequential Scan, Index Scan), стоимость операций, количество строк. Используется для: оптимизации медленных запросов, проверки использования индексов, понимания узких мест. EXPLAIN показывает план, EXPLAIN ANALYZE выполняет и показывает реальное время. Ключевые метрики: cost, rows, execution time.

131. Что такое deadlock в БД и как его избежать?

Deadlock — ситуация, когда две или более транзакции взаимно блокируют друг друга, ожидая освобождения ресурсов. Пример: T1 блокирует A, ждет B; T2 блокирует B, ждет A. БД автоматически обнаруживает (timeout или граф ожидания) и откатывает одну транзакцию. Предотвращение: всегда блокировать ресурсы в одинаковом порядке, использовать короткие транзакции, минимизировать блокировки, использовать оптимистичные блокировки (versioning), установить lock timeout.

132. Что такое оптимистичная и пессимистичная блокировки?

Пессимистичная блокировка — блокирует данные при чтении до конца транзакции (SELECT FOR UPDATE). Предотвращает изменения другими транзакциями, гарантирует согласованность, но снижает конкурентность, риск deadlock. Оптимистичная — предполагает отсутствие конфликтов, проверяет перед записью (version/timestamp). Если данные изменились — откат и retry. Лучше для read-heavy, низкой конкуренции на запись. Пессимистичная для высокой конкуренции, критичных операций. Компромисс: блокировки vs производительность.

133. Что такое MVCC?

MVCC (Multi-Version Concurrency Control) — механизм управления конкурентным доступом через версионирование данных. Каждая транзакция видит snapshot данных на момент начала. При изменении создается новая версия строки, старая сохраняется для других транзакций. Преимущества: читатели не блокируют писателей, писатели не блокируют читателей, высокая конкурентность. Используется в PostgreSQL, MySQL InnoDB, Oracle. Недостатки: overhead на хранение версий, требуется vacuum/purge для очистки старых версий.

134. Что такое Write-Ahead Logging (WAL)?

WAL (Write-Ahead Logging) — техника для обеспечения durability и atomicity. Принцип: изменения сначала записываются в лог (последовательная запись, быстро), затем применяются к данным (случайная запись, медленнее). При сбое: replay WAL для восстановления незафиксированных изменений. Преимущества: быстрые транзакции (отложенная запись данных), crash recovery, основа для репликации (streaming WAL). В PostgreSQL — WAL, MySQL — redo log, SQLite — journal. Компромисс: дополнительное I/O, но критично для надежности.

135. Что такое партиционирование (partitioning) таблиц?

Партиционирование — разделение большой таблицы на меньшие части (партиции) по правилу. Типы: Range (диапазоны значений, например даты), List (список значений), Hash (хеш функция), Composite (комбинация). Преимущества: быстрее запросы (сканируются только нужные партиции), легче удаление старых данных (drop partition), параллельные операции, улучшенное обслуживание (VACUUM, ANALYZE). Partition pruning — оптимизатор исключает ненужные партиции. Используется для больших таблиц (логи, временные ряды).

SOLID принципы

136. Что такое SOLID принципы и зачем они нужны?

SOLID — пять принципов дизайна для поддерживаемого кода: **Single Responsibility**, **Open/Closed**, **Liskov Substitution**, **Interface Segregation**, **Dependency Inversion**. Цели: меньше связанности (coupling), выше связность (cohesion), проще тестировать и менять. В Go реализуются через интерфейсы, композицию и маленькие пакеты — не через наследование.

137. Single Responsibility Principle (SRP)

Каждый модуль/тип — одна зона ответственности, одна причина для изменения. Нарушение: `User` валидирует, пишет в БД и шлёт email. Правильно: `User` — данные, `Validator`, `UserRepository`, `EmailService` — отдельные типы. Проще тестировать и менять независимо.

138. Open/Closed Principle (OCP)

Код открыт для расширения, закрыт для модификации. В Go: новый функционал — новая реализация интерфейса, а не новый `if/switch`. Пример: `PaymentProcessor` с реализациями `CardPayment`, `CryptoPayment` — добавление метода не трогает существующий код.

139. Liskov Substitution Principle (LSP)

Подтип должен быть заменяем базовым типом без сюрпризов для клиента. Нарушение: `ReadOnlyRepo` с `Save()`, который паникует. Правильно: узкий интерфейс `Reader / Writer` — клиент зависит только от того, что реально использует. В Go интерфейсы маленькие — LSP про контракт поведения, не про иерархию классов.

140. Interface Segregation Principle (ISP)

Клиент не должен зависеть от методов, которые не использует. Лучше несколько маленьких интерфейсов (`Reader`, `Writer`, `Closer`), чем один «толстый» `ReadWriteCloser` везде. В Go интерфейсы определяются потребителем — это естественный ISP.

141. Dependency Inversion Principle (DIP)

Зависимость от абстракций, не от конкретики. Высокоуровневый код зависит от интерфейса `UserStore`, реализация `PostgresUserStore` инжектится снаружи (конструктор, `wire`, `fx`). Упрощает тесты: подставить mock. Антипаттерн: `import` конкретного пакета БД прямо в handler.

142. Как SOLID помогает в тестировании и какие компромиссы?

Тестирование: DIP → mock интерфейсов; SRP → unit-тесты на один слой; ISP → меньше stub-методов. Компромиссы: избыточные интерфейсы на один тип усложняют навигацию; «идеальный» SOLID на 3 строки кода — over-engineering. Практика: интерфейс там, где есть ≥2 реализации или нужен mock; иначе конкретный тип.

143. Что такое Apache Kafka и для чего используется?

Kafka — распределённый брокер сообщений (distributed commit log). Сообщения пишутся в топик и хранятся на диске заданное время (retention), а не удаляются сразу после чтения. Используется для: event streaming (потоки событий между сервисами), асинхронной обработки, CDC (Change Data Capture, например Debezium), логов аналитики, буферизации пиков нагрузки. Высокая пропускная способность, горизонтальное масштабирование через партиции, отказоустойчивость через репликацию.

144. Из каких компонентов состоит Kafka?

Broker — сервер, хранящий данные топиков. **Topic** — именованный поток сообщений. **Partition** — единица параллелизма и хранения внутри топика. **Producer** — публикует сообщения. **Consumer** — читает сообщения. **Consumer group** — группа консьюмеров, совместно обрабатывающих топик (каждая партиция — одному консьюмеру группы). **ZooKeeper** (старые версии) или **KRaft** (Kafka 3.3+, по умолчанию с 4.0) — координация кластера, метаданные, выбор лидеров. **Schema Registry** (опционально, Confluent) — хранение схем Avro/Protobuf/JSON.

145. Что такое topic, partition и offset?

Topic — логическая категория сообщений (например `orders.created`). **Partition** — физический упорядоченный лог внутри топика; сообщения в партиции имеют строгий порядок. Партиции распределены по брокерам — это даёт параллелизм. **Offset** — монотонно растущий номер сообщения внутри партиции (не глобальный). Консьюмер отслеживает offset — «до какого места прочитал». Offset коммитится в специальный топик `__consumer_offsets` или во внешнее хранилище.

146. Как обеспечивается порядок сообщений в Kafka?

Порядок гарантирован **только внутри одной партиции**. Если нужен порядок для сущности (заказ, пользователь) — все её события должны попадать в одну партицию через **partition key** (producer указывает ключ, хеш ключа → номер партиции). Больше партиций = больше параллелизма, но порядок между партициями не гарантируется. Для глобального порядка — одна партиция (узкое место).

147. Что такое consumer group и как работает rebalance?

Consumer group — набор консьюмеров с одним `group.id`, совместно читающих топик. Каждая партиция назначается **ровно одному** консьюмеру группы. При добавлении/удалении консьюмера или изменении партиций происходит **rebalance** — перераспределение партиций. Протоколы: Range, RoundRobin, Cooperative (sticky) — последний минимизирует «stop-the-world» при rebalance. Во время rebalance обработка останавливается — частая причина лагов. `max.poll.interval.ms` и `session.timeout.ms` влияют на детект «мёртвых» консьюмеров.

148. Как устроена репликация в Kafka? Что такое ISR?

Каждая партиция имеет **replication factor** копий на разных брокерах. Одна реплика — **leader** (принимает read/write), остальные — **followers** (реплицируют). **ISR** (In-Sync Replicas) — реплики, не отстающие от лидера больше чем на `replica.lag.time.max.ms`. Запись считается подтверждённой при `acks=all` (или `-1`) когда все ISR записали. При падении лидера новый выбирается из ISR. `min.insync.replicas` — минимум ISR для записи; если меньше — producer получит ошибку (защита от потери данных).

149. Чем отличаются at-least-once, at-most-once и exactly-once?

At-most-once — сообщение может потеряться, но не продублируется. Producer: `acks=0` или `fire-and-forget`; consumer коммитит offset до обработки. **At-least-once** — сообщение не теряется, но может обработаться повторно. Producer ждёт ack; consumer коммитит offset **после** обработки. Нужна идемпотентность на стороне обработчика. **Exactly-once** — каждое сообщение обрабатывается ровно один раз. В Kafka: идемпотентный producer + транзакции (read-process-write в одной транзакции) или EOS в Kafka Streams. Дороже по latency и сложнее в настройке.

150. Что такое идемпотентный producer?

Producer с `enable.idempotence=true` получает уникальный **Producer ID (PID)** и для каждой партиции ведёт счётчик sequence number. Брокер отбрасывает дубликаты при ретрайх (сетевой сбой → повторная отправка). Гарантирует отсутствие дублей **внутри одной сессии producer** на уровне записи в партицию. Не заменяет идемпотентность consumer — если сообщение уже обработано, а offset не закоммитен, consumer прочтёт его снова. Требуется `acks=all`, `retries > 0`, `max.in.flight.requests.per.connection <= 5`.

151. Как работают Kafka transactions?

Транзакции позволяют атомарно: записать в несколько партиций/топиков + закоммитить consumer offsets в одной транзакции. Producer вызывает `initTransactions()` → `beginTransaction()` → `send/consumer offsets` → `commitTransaction()` или `abortTransaction()`. Брокер пишет маркеры транзакций (control records). Консьюмер с `isolation.level=read_committed` видит только закоммитенные сообщения. Используется для exactly-once семантики в пайплайнах обработки. Overhead: координатор транзакций, таймаут `transaction.timeout.ms`.

152. В чем разница между Kafka и RabbitMQ?

Kafka — log-based, сообщения хранятся (retention), consumer сам тянет (pull), высокий throughput, replay (перечитать с любого offset). **RabbitMQ** — queue-based, сообщение удаляется после ack (по умолчанию), push-модель, гибкая маршрутизация (exchanges, routing keys), лучше для task queues и RPC. Kafka — event streaming, аналитика. RabbitMQ — классические очереди задач, сложная маршрутизация, меньшие объёмы. В Go: `segmentio/kafka-go`, `confluent-kafka-go` (librdkafka) для Kafka; `amqp091-go` для RabbitMQ.

153. Что такое KRaft и чем он заменил ZooKeeper?

KRaft (Kafka Raft) — встроенный механизм консенсуса на базе Raft для хранения метаданных кластера (топики, партиции, ACL, конфигурация). Заменяет внешний **ZooKeeper**, который раньше был обязательной зависимостью. Плюсы: проще деплой (меньше

компонентов), быстрее failover контроллера, масштабирование до миллионов партиций. С Kafka 4.0 KRaft — единственный режим (ZooKeeper удалён). Миграция: rolling upgrade с режима ZK на KRaft.

154. Что такое consumer lag и как с ним бороться?

Lag — разница между последним offset в партиции (log end offset) и offset, закоммиченным consumer group. Растущий lag = consumer не успевает. Причины: медленная обработка, rebalance, мало партиций (мало параллелизма), GC/long pauses, `max.poll.records` слишком большой. Решения: увеличить партиции и инстансы consumer (до числа партиций), оптимизировать handler, batch-обработка, асинхронная обработка с отдельным коммитом, мониторинг через Kafka Exporter / Burrow / встроенные метрики. Lag по партиции — признак неравномерного распределения ключей (hot partition).

Docker и Kubernetes

155. Чем контейнер отличается от виртуальной машины?

VM эмулирует железо: на гипервизоре свой guest OS, ядро, драйверы — тяжело (гигабайты, минуты на старт). **Контейнер** делит ядро хоста, изолирует процессы через namespaces и cgroups — легче (мегабайты, секунды). VM — сильная изоляция, разные ОС. Контейнер — изоляция процессов, одна ОС, идеален для микросервисов. Go-бинарник + минимальный образ — типичный деплой.

156. Что такое image и container?

Image — неизменяемый шаблон (слои файловой системы + метаданные: CMD, ENV, EXPOSE). **Container** — запущенный экземпляр image: процесс + writable layer поверх read-only слоёв. Один image → много контейнеров. Image собирается из Dockerfile, хранится в registry (Docker Hub, GitLab Registry, ECR).

157. Как устроен Dockerfile? Что такое слои и cache?

Dockerfile — инструкции сборки image. Каждая инструкция (FROM, RUN, COPY) создаёт **слой**. Слои кешируются: если инструкция и контекст не изменились — берётся из cache. Порядок важен: редко меняющиеся шаги (зависимости) — выше, часто меняющиеся (код) — ниже. COPY перед RUN go build — пересборка только при изменении кода.

158. CMD VS ENTRYPOINT, COPY VS ADD

ENTRYPOINT — фиксированная точка входа контейнера, не перезаписывается полностью аргументами docker run. **CMD** — аргументы по умолчанию, легко переопределяются. Паттерн: ENTRYPOINT ["/app"] + CMD ["--config", "prod.yaml"]. **COPY** — копирует файлы из контекста сборки (рекомендуется). **ADD** — то же + auto-распаковка tar и загрузка по URL (используй редко).

159. Volume vs bind mount — когда что

Volume — управляемое Docker хранилище, живёт отдельно от контейнера, бэкапится, переносится между хостами. **Bind mount** — примонтированная директория хоста (/host/path:/container/path), удобно для dev (hot reload). Volume — для prod данных (БД, файлы). Bind mount — для разработки и конфигов с хоста. Оба переживают перезапуск контейнера.

160. Как уменьшить размер образа Go-приложения?

Статическая сборка: CGO_ENABLED=0 go build -ldflags="-s -w". Базовый image: scratch (пустой) или distroless (минимум libc). Не класть в image: исходники, тесты, toolchain. Multi-stage build: builder с golang — compile, runtime — только бинарник. Итог: 10–20 MB вместо 800 MB на golang:latest.

161. Что такое multi-stage build и зачем он Go-разработчику?

Несколько FROM в одном Dockerfile. Стадия 1: golang:1.24 — компиляция. Стадия 2: scratch/alpine — копируем только бинарник COPY --from=builder /app/main . . В финальном image нет компилятора, исходников, кеша модулей. Меньше attack surface, быстрее pull в k8s.

162. Docker Compose — зачем нужен

Оркестрация нескольких контейнеров на одной машине для **локальной разработки**: app + PostgreSQL + Redis + Kafka одной командой docker compose up. Описание в docker-compose.yml: сервисы, сети, volumes, env. Не замена k8s в prod, но стандарт для dev/stage окружений.

163. Как пробросить переменные окружения и секреты в контейнер

ENV в Dockerfile — значения по умолчанию (не для секретов). docker run -e VAR=value или env_file: в compose. **Secrets** в compose/k8s — отдельно от image, монтируются как файлы (/run/secrets/db_password). Никогда не bake секреты в image. В k8s — Secret + env или volume mount.

164. Почему в контейнере не должно быть root-пользователя

Root в контейнере = root на хосте при escape из namespace (уязвимость ядра, misconfig). **USER** в Dockerfile — запуск от non-root. В k8s: securityContext.runAsNonRoot: true. Меньше рисков, требование security compliance. Go-приложение слушает порт >1024 или использует capabilities.

165. Зачем нужен Kubernetes? Что он даёт поверх Docker

Docker запускает контейнеры на одной машине. **Kubernetes** — оркестратор кластера: деплой на N нод, self-healing (перезапуск упавших pod), масштабирование, rolling updates, service discovery, балансировка, secrets/config. Декларативно: описываешь желаемое состояние, k8s приводит к нему.

166. Pod, Deployment, Service — что делает каждый

Pod — минимальная единица: один или несколько контейнеров с общей сетью/volume, ephemeral. **Deployment** — декларативное управление ReplicaSet: желаемое число pod, rolling update, rollback. **Service** — стабильный сетевой endpoint (ClusterIP/NodePort/LoadBalancer) для доступа к pod за балансировщиком. Типичный стек: Deployment → Pod → Service.

167. ReplicaSet vs Deployment vs StatefulSet

ReplicaSet — поддерживает N идентичных pod (заменён Deployment как API для пользователя). **Deployment** — ReplicaSet + стратегии обновления. **StatefulSet** — для stateful: стабильные имена pod (app-0, app-1), стабильные volume, упорядоченный старт/стоп. БД, Kafka, ZooKeeper — StatefulSet. Stateless Go API — Deployment.

168. ConfigMap и Secret — зачем и чем отличаются

ConfigMap — несекретная конфигурация (URLs, feature flags, yaml). **Secret** — чувствительные данные (пароли, токены), base64 в etcd (не шифрование по умолчанию!). Монтируются как env или файлы в pod. Отделяют конфиг от image — один image на все окружения. Secret — с RBAC и encryption at rest в prod.

169. Ingress — что это и зачем

Ingress — L7 маршрутизация HTTP/HTTPS в кластер: домены, пути, TLS termination. Нужен Ingress Controller (nginx, traefik). Внешний трафик → LoadBalancer → Ingress → Service → Pod. Альтернатива: Service type LoadBalancer на каждый сервис (дорого в облаке).

170. Liveness vs Readiness probe

Liveness — жив ли контейнер. Fail → kubelet **перезапускает** pod (deadlock, panic loop). **Readiness** — готов ли принимать трафик. Fail → pod **убирается из Service endpoints**, но не перезапускается (прогрев кеша, миграции, зависимость от БД). Go: liveness — /healthz, readiness — /ready с проверкой БД.

171. Rolling update и rollback

Rolling update — постепенная замена pod: maxSurge (сколько лишних pod), maxUnavailable (сколько можно убить). Zero-downtime при readiness probe. **Rollback** — kubectl rollout undo deployment/app к предыдущей ReplicaSet. История в kubectl rollout history. Стратегия Recreate — убить все, поднять новые (даунтайм, редко).

172. Что такое requests/limits (CPU, memory)

requests — гарантированные ресурсы для scheduling (на какую ноду поместить pod). **limits** — максимум: CPU throttling, OOMKill при превышении memory. Go: типично requests: 100m CPU, 128Mi / limits: 500m, 512Mi — подбирать по pprof. Без limits — noisy neighbor. Без requests — scheduler кладёт куда попало.

173. Что происходит, когда pod падает

Kubelet детектит (probe fail, exit code != 0) → удаляет pod → Deployment controller создаёт новый pod на доступной ноде → readiness OK → Service добавляет в endpoints. Данные в ephemeral container layer теряются — state в volume или внешней БД. Grace period: SIGTERM → graceful shutdown → SIGKILL через terminationGracePeriodSeconds.

174. Namespace — зачем

Логическое разделение ресурсов в кластере: dev, staging, prod, team-billing. Изоляция RBAC, квот, сетевых политик. Не изоляция на уровне железа — один кластер. kubectl get pods -n prod. По умолчанию — default.

175. HPA — горизонтальное автомасштабирование (базово)

Horizontal Pod Autoscaler — меняет replicas Deployment по метрикам: CPU, memory, custom (RPS из Prometheus). minReplicas — maxReplicas. Пример: CPU > 70% → добавить pod. Требуется metrics-server или custom metrics adapter. Не путать с VPA (вертикальное — размер pod).

176. Как Go-сервис деплоится в k8s: healthcheck, graceful shutdown, SIGTERM

- 1) **Dockerfile** multi-stage → маленький image.
- 2) **Deployment** + probes на /healthz и /ready.
- 3) **SIGTERM** при остановке pod — signal.Notify + server.Shutdown(ctx) с таймаутом < terminationGracePeriodSeconds.
- 4) **PreStop hook** (опционально) — sleep для drain из Service.
- 5) **ConfigMap/Secret** для конфига.
- 6) **resources** по результатам нагрузочного теста.

177. Что такое gRPC и чем отличается от REST?

gRPC — RPC-фреймворк поверх HTTP/2 с **Protocol Buffers** (бинарная сериализация). REST — обычно JSON over HTTP/1.1, ресурсо-ориентированный. gRPC: строгий контракт (.proto), быстрее и компактнее, bidirectional streaming, встроенные deadlines. REST: проще отладка (curl), шире поддержка браузеров. В микросервисах внутри кластера — часто gRPC; наружу — REST/GraphQL.

178. Что такое Protocol Buffers (protobuf)?

Язык-независимый формат описания структур данных в .proto файлах. protoc генерирует Go-структуры и сериализацию. Бинарный, компактный, быстрый parse. Схема версионизируется: не удалять/не менять номера полей, добавлять новые с новыми tag. Альтернативы: JSON Schema, Avro. В Go: google.golang.org/protobuf, кодоген через `protoc-gen-go-grpc`.

179. Какие типы вызовов есть в gRPC?

- 1) **Unary** — один запрос, один ответ (как обычный RPC).
- 2) **Server streaming** — клиент шлёт один запрос, сервер — поток ответов.
- 3) **Client streaming** — клиент шлёт поток, сервер — один ответ.
- 4) **Bidirectional streaming** — оба потока одновременно. Streaming для больших данных, real-time, без буферизации всего в память.

180. Как работают deadline и cancellation в gRPC?

Клиент задаёт deadline: `ctx, cancel := context.WithTimeout(ctx, 3*time.Second)`. Deadline передаётся в metadata по HTTP/2. Сервер проверяет `ctx.Done()` — отмена при таймауте или cancel клиента. В Go gRPC deadline/cancellation интегрирован с `context.Context`. Важно: прокидывать ctx во все downstream вызовы и БД-запросы.

181. Как обрабатывать ошибки в gRPC?

Статус через `status.Error(codes.NotFound, "user not found")`. Коды: OK, InvalidArgument, NotFound, AlreadyExists, DeadlineExceeded, Unavailable, Internal. Клиент: `status.FromError(err) → code + message`. **Не** возвращать stack trace клиенту. Retry только на Unavailable, DeadlineExceeded с backoff. В Go: google.golang.org/grpc/status, codes.

182. Что такое interceptors (middleware) в gRPC?

Аналог HTTP middleware: цепочка до/после handler. **UnaryInterceptor** — логирование, auth, метрики, recovery от panic, tracing. Server: `grpc.UnaryInterceptor(loggingInterceptor)`. Client: для retry, tracing propagation. Несколько interceptors — `grpc.ChainUnaryInterceptor(a, b, c)`.

183. gRPC vs REST — когда что выбрать?

gRPC: внутренние микросервисы, high throughput, streaming, строгий контракт, polyglot (много языков). **REST**: публичный API, мобильные/веб-клиенты, простота, кеширование HTTP. **Оба**: gRPC-gateway генерирует REST из .proto. Go-бэкенд часто: gRPC внутри, REST наружу.

184. Как тестировать gRPC-сервисы в Go?

- 1) **In-process**: `bufconn` — listener в памяти, без сети.
- 2) `grpc.NewClient` на тестовый сервер.
- 3) Моки интерфейсов сервисного слоя (не gRPC напрямую).
- 4) Интеграционные тесты с реальным сервером на случайном порту. Проверять status codes, metadata, deadline behaviour.

Observability (метрики, логи, трейсы)

185. Что такое observability и из чего она состоит?

Способность понять внутреннее состояние системы по внешним сигналам. Три столпа: **метрики** (числа во времени), **логи** (события с контекстом), **трейсы** (путь запроса через сервисы). Плюс **профилирование** (pprof) для глубокого анализа. Цель: быстро отвечать «почему медленно / почему упало» в production.

186. Чем метрики отличаются от логов?

Метрики — агрегированные числа: RPS, latency p99, error rate, CPU. Дешёвые в хранении, хороши для алертов и дашбордов. **Логи** — дискретные события: «заказ 123 создан», stack trace. Дороже в объёме, нужны для детальной отладки. Правило: алертить по метрикам, расследовать по логам и трейсам.

187. Что такое RED и USE методологии?

RED (для сервисов): **R**ate (запросов/сек), **E**rrors (доля ошибок), **D**uration (латентность). **USE** (для ресурсов): **U**tilization (% CPU/диска), **S**aturation (очередь, ожидание), **E**rrors. Минимальный набор метрик для любого Go HTTP/gRPC сервиса: `http_requests_total`, `http_request_duration_seconds`, `error_counter` по status code.

188. Как устроен Prometheus и pull-модель?

Prometheus **скрейпит** (pull) `/metrics` endpoint каждые N секунд. Метрики в формате text exposition. Хранит time series в TSDB. **Alertmanager** — алерты по правилам. В Go: `prometheus/client_golang` — Counter, Gauge, Histogram, Summary. Histogram для latency с buckets. Grafana — визуализация. Push — только для batch jobs (Pushgateway).

189. Какие метрики собирать у Go-сервиса?

Обязательно: RPS, latency (histogram p50/p95/p99), error rate по endpoint. Runtime: `go_goroutines`, `go_memstats_*`, GC pause — из `prometheus/client_golang` collectors. Бизнес-метрики: заказы/сек, размер очереди. Не раздувать cardinality: не label'ить `user_id` на каждый запрос — взрыв time series.

190. Что такое distributed tracing?

Отслеживание одного запроса через цепочку сервисов. **Trace** — весь путь, **span** — один участок (HTTP call, DB query). **Trace ID** прокидывается через заголовки (`tracetransparent`, Jaeger). В Go: OpenTelemetry SDK → Jaeger/Tempo. Показывает: где потратили 2 из 3 секунд, какой сервис тормозит.

191. Структурированное логирование в Go

Текстовые логи плохо парсятся. **Структурированные** — JSON/key-value: `slog.Info("order created", "order_id", id, "user_id", uid)`. Поля фильтруются в Loki/ELK. Стандарт: `log/slog` (Go 1.21+). Обязательно: `request_id` / `trace_id` в каждой строке. Уровни: Debug, Info, Warn, Error. Не логировать секреты и PII.

192. Как профилировать Go-сервис в production?

`net/http/pprof` на admin-порту (не публичном): `/debug/pprof/profile` (CPU), `/debug/pprof/heap`. Снять: `go tool pprof http://localhost:6060/debug/pprof/profile?seconds=30`. Continuous profiling: Pyroscope, Grafana Phlare. Метрики + трейсы находят «что медленно», pprof — «почему в коде». Включать с низким overhead, защищать auth/network policy.

Сценарии и troubleshooting

Вопросы формата «есть проблема — расскажи, как решать». На себе ждут пошаговый план, а не угадывание одной причины.

193. Latency выросла после деплоя — с чего начать?

1. **Сопоставить время** — графики RPS, p50/p95/p99, error rate в Grafana; совпадает ли с деплоем/изменением конфига.
2. **Откатить или канареечный pod** — если явная корреляция, быстро проверить гипотезу rollback'ом.
3. **Трейсы** — один медленный запрос: какой span съел время (БД, внешний API, сериализация).
4. **pprof** — CPU profile, сравнить с прошлой версией.
5. **БД** — slow query log, pg_stat_activity, пост locks.
6. **Diff деплоя** — новая зависимость, убранный индекс, изменённый timeout, feature flag.
7. **Нагрузка** — может latency выросла из-за роста RPS, а не из-за кода.

194. Растёт число горутин (goroutine leak) — как найти?

1. Метрика go_goroutines в Prometheus — тренд вверх без спада после пика нагрузки.
2. **pprof goroutine**: go tool pprof http://localhost:6060/debug/pprof/goroutine — stack trace всех горутин, ищем одинаковые стеки (застыли на chan receive, sync.Mutex.Lock, time.Sleep).
3. Типичные причины: горутина ждёт закрытия канала, который никто не закроет; забытый ctx без cancel; worker без exit condition.
4. Локально: goLeak в тестах.
5. Фикс: context с cancel, закрытие каналов отправителем, таймауты на блокирующие операции.

195. Pod в k8s убит OOMKilled — что делать?

1. kubectl describe pod — Last State: Terminated, Reason: OOMKilled.
2. Контейнер превысил memory limit.
3. **pprof heap** до рестарта (если успели снять) — что съело память: рост слайсов, кеш без eviction, утечка через global map.
4. Проверить limits vs requests — может limit слишком низкий для нормальной работы.
5. Go: GC не успевает при spike аллокаций; проверить GOGC, размер буферов, загрузку больших тел в память целиком.
6. Краткосрочно: поднять limit. Долгосрочно: профилировать heap, streaming вместо буферизации всего ответа.

196. Паника в горутине — что будет с сервисом?

Необработанная panic в горутине **убивает только эту горутину**, не весь процесс (в отличие от panic в main). Но: запрос оборвётся, ресурсы могут не освободиться, воркер исчезнет из пула.

Решение:

1. defer recover() в HTTP middleware и в worker goroutines — логировать stack, возвращать 500.
2. net/http уже ловит panic в handler.
3. В production: alert на panic в логах, stack trace в логере.
4. Не использовать recover для обычных ошибок — только для truly unexpected.

197. Connection pool к PostgreSQL исчерпан — почему и как починить?

Симптом: pq: sorry, too many clients already, таймауты на db.Query.

Причины:

1. **Утечка соединений** — rows/stmt не закрыты (defer rows.Close()).
2. **Слишком много реплик** сервиса × MaxOpenConns > лимит Postgres.
3. **Долгие транзакции** держат conn.
4. **Медленные запросы** — conn заняты, пул исчерпан.

Решение: проверить pg_stat_activity, закрывать rows, уменьшить MaxOpenConns на инстанс, PgBouncer, оптимизировать запросы, таймауты через context.

198. Клиент дважды отправил оплату — как защититься?

Идемпотентность.

1. Клиент шлёт Idempotency-Key: uuid в заголовке.
2. Сервер: таблица idempotency_keys(key, response, status) — при повторе вернуть сохранённый ответ.
3. В Kafka: processed_events(event_id) с unique constraint.
4. В БД: UNIQUE(order_id) + проверка статуса перед оплатой.

At-least-once доставка требует идемпотентной обработки на стороне consumer.

199. Два пользователя бронируют последнее место одновременно

Race condition без блокировки — оба прочитают seats > 0, оба запишут.

1. **Пессимистично**: SELECT ... FOR UPDATE в транзакции — второй ждёт, увидит 0 мест.
2. **Оптимистично**: UPDATE ... WHERE count > 0 — проверить RowsAffected == 1, иначе retry.
3. **Redis**: DECR атомарно.

Выбор: высокая конкуренция на одну строку → FOR UPDATE или атомарный UPDATE; read-heavy → optimistic.

200. Thundering herd при протухании кеша

Тысяча запросов одновременно промахнулись в Redis — все пошли в БД.

1. **Jitter TTL** — разброс времени протухания (TTL + random(0, 60s)).
2. **Singleflight** (golang.org/x/sync/singleflight) — один запрос на ключ идёт в БД, остальные ждут.
3. **Probabilistic early refresh** — обновлять кеш до протухания с малой вероятностью.

4. **Mutex per key** в приложении.

В Go: `singleflight.Group` — стандартный паттерн.

201. Kafka consumer lag растёт — план действий

1. Lag по партициям — одна партиция (hot key) или все (мало consumers).
2. **Медленный handler** — pprof, время обработки одного сообщения, блокировки на БД.
3. **Rebalance** в логах — частые stop/start consumer group.
4. **Мало партиций** — увеличить партиции и инстансы consumer (до числа партиций).
5. **max.poll.interval.ms** — handler дольше интервала → consumer исключается из группы.
6. Batch processing + commit после batch.
7. Масштабировать consumers горизонтально.

202. Дубликаты сообщений из Kafka — как обрабатывать?

Kafka гарантирует at-least-once при retry. Consumer **должен быть идемпотентным**:

1. Хранить `event_id` в БД, `INSERT ... ON CONFLICT DO NOTHING`.
2. Проверка «уже обработано» перед side effect.
3. Сначала запись в БД (outbox/processed), потом side effect — или компенсация.
4. Для платежей — бизнес-ключ уникален в таблице.

Exactly-once сложно — проще идемпотентный handler.

203. 502/503 под нагрузкой — где искать

1. **Ingress/LB timeout** < время ответа сервиса — поднять timeout или ускорить сервис.
2. **Исчерпаны воркеры** — все goroutines заняты, очередь растёт.
3. **Upstream недоступен** — circuit breaker открыт, 503.
4. **k8s**: pod не ready, endpoints пустые.
5. **File descriptors** — лимит исчерпан.

Дальше: метрики RPS vs latency vs errors, трейс одного 502 запроса, load test.

204. Rolling update в k8s, но был даунтайм

Причины:

1. **Readiness probe** не проверяет реальную готовность (БД недоступна — pod в endpoints, но падает).
2. **maxUnavailable: 100%** или `replicas: 1`.
3. **PreStop hook** отсутствует — pod убит до drain из Service.
4. **Graceful shutdown** не реализован — SIGTERM, но процесс убит с активными запросами.

Фикс: readiness с проверкой зависимостей, `maxUnavailable: 0`, `maxSurge: 1`, `server.Shutdown`.

205. Сервис не подключается к БД после деплоя в k8s

1. **DNS** — hostname БД резолвится из pod (`nslookup`).
2. **Secret/ConfigMap** — правильные credentials, смонтированы в pod.
3. **NetworkPolicy** — egress к порту 5432 разрешён.
4. **Service name** — `postgres:5432` vs внешний host.
5. **SSL mode** — `sslmode=require` в prod.
6. Логи при старте — `db.Ping()` в readiness.

Локально работало через `localhost` — в k8s нужен cluster DNS.

206. N+1 запросов к БД — как обнаружить и исправить

Симптом: один HTTP-запрос → 100 SQL-запросов.

Обнаружение: query count per request, `pg_stat_statements`, трейс с N одинаковыми SELECT.

Типично: цикл `for _, user := range users { db.GetOrders(user.ID) }`.

Фикс:

1. `WHERE user_id IN (...)` одним запросом или JOIN.
2. DataLoader / batch load.
3. ORM: Preload в GORM.
4. В Go: один запрос + `map[userID][]Order` в памяти.

207. Как спроектировать rate limiter для публичного API?

Требования: лимит N req/min per API key / IP, 429 при превышении.

1. **In-memory** token bucket — просто, но не работает при N репликах без синхронизации.
2. **Redis** — `INCR` + `EXPIRE` или sliding window; общий лимит на кластер.
3. **API Gateway** (nginx, Kong) — лимит до приложения.

Middleware в Go: проверка до handler, заголовки `X-RateLimit-Remaining`. Учесть burst vs sustained rate.

208. Миграция с монолита на микросервисы без даунтайма (Strangler)

1. **Strangler Fig** — новый сервис через проxy/gateway, старый — остальное.
2. **Dual write** или **CDC (Debezium)** — синхронизация данных.
3. **Сравнение ответов** — shadow traffic, diff старого и нового API.
4. Постепенно переключать % трафика (feature flag, canary).

5. Откат всегда возможен.
6. Не делать big bang.

Для Go: отдельный сервис, общая БД на первом этапе допустима, потом выделение schema.

209. Медленный endpoint — пошаговая оптимизация

1. **Замерить** — benchmark, pprof, trace: CPU vs I/O bound.
2. **БД** — EXPLAIN, индексы, N+1, лишние поля.
3. **Кеш** — Redis для read-heavy, TTL, инвалидация.
4. **Аллокации** — `go test -bench -benchmem`, `sync.Pool` для буферов.
5. **Конкурентность** — параллельные вызовы через `errgroup`.
6. **Пагинация** вместо «вернуть всё».
7. Повторить замер — одно узкое место за раз, не гадать.

Magnit Tech

Magnit Tech: atomic в Go

M1. Как работают atomic операции под капотом в Go?

Уровень CPU

Атомарность — одна неделимая машинная инструкция, а не «мьютекс спрятанный в int». На x86/arm64:

- LOCK CMPXCHG — compare-and-swap (CAS): «если значение всё ещё X — запиши Y»
- XADD — атомарное сложение
- SWAP — атомарная замена

`atomic.AddInt64(&v, 1)` компилируется в такую инструкцию. Другой поток не увидит промежуточное состояние «прочитал 5 → прибавил 1 → ещё не записал».

Почему `count++` не атомарен

В Go `count++` — три шага: `load` → `add` → `store`. Две горутины могут обе прочитать `10`, обе записать `11` — один инкремент потерян. Это `data race` (`go test -race`).

Память и happens-before

`sync/atomic` даёт гарантию видимости между горутинами на **одной** переменной: `Store` в одной горутине «виден» `Load` в другой. Обычные `read/write` без `atomic` компилятор и CPU могут переупорядочить — для флага «инициализация завершена» нужен `atomic` или `mutex`.

API в Go

Низкоуровневые: `atomic.LoadInt64`, `StoreInt64`, `AddInt64`, `CompareAndSwapInt64`, `SwapInt64`, `uintptr`, `unsafe.Pointer`.

Обёртки (Go 1.19+): `atomic.Int64`, `atomic.Bool`, `atomic.Pointer[T]`, `atomic.Value`.

Ограничения

Atomic защищает **одну ячейку**. Инвариант на двух полях (`reserved <= quantity`) одним atomic не обеспечить — нужен `mutex` или транзакция в БД.

Подходит: счётчики метрик, флаг shutdown, lock-free head списка, token bucket **в одном процессе**.

Atomic vs Mutex

	Atomic	Mutex
Скорость	Быстрее на простых ops	Дороже (contention, futex)
Сложность	Только простые операции	Любой инвариант под lock
Ошибки	Легко сломать логику CAS-loop	Проще reasoning

На себе Magnit часто спрашивают: «почему не `int` в счётчике RPS» — см. [06-rate-limiter.md](#).

Magnit Tech: panic и error

M2. Как работают panic в Go и чем отличаются от error?

Error — обычное значение

```
func readConfig(path string) (*Config, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return nil, fmt.Errorf("read config: %w", err)
    }
    // ...
}
```

- Возвращается явно, обрабатывается `if err != nil`
- Не раскручивает стек
- Ожидаемые сбои: файл не найден, таймаут, валидация

Panic — аварийная остановка горутины

```
panic("invariant violated")
```

- Раскручивает стек, выполняет **defer** по пути
- Если нет `recover` — **горутина падает** (main упадёт, HTTP handler в net/http ловится middleware)
- Для «это невозможно» / баг программиста, не для бизнес-ошибок

recover

```
func handler(w http.ResponseWriter, r *http.Request) {
    defer func() {
        if v := recover(); v != nil {
            log.Printf("panic: %v\n%s", v, debug.Stack())
            http.Error(w, "internal error", 500)
        }
    }()
    // ...
}
```

`recover()` работает **только в defer** той же горутины, где был panic.

Ключевые отличия

	error	panic
Когда	Ожидаемый сбой	Баг / невозстановимо в месте вызова
Контроль	Вызывающий решает	Раскрутка стека или recover
Горутина	Продолжает работу	Умирает без recover
В HTTP API	4xx/5xx в ответе	500 + лог (если поймали)

Panic в горутине

Panic в **дочерней** горутине **не убивает** main и другие горутины — только её. Сервис «жив», но запрос/воркер оборван, возможна утечка ресурсов. В production: recover в middleware и worker pool.

Когда что на себе

- Валидация входа, БД недоступна, 404 → **error**
- `nil` pointer по логике «не бывает», index out of range после бага → можно panic в dev, в prod лучше error + метрика
- **Не** использовать panic для flow control

Magnit Tech: уровни изоляции

М3. Примеры аномалий на разных уровнях изоляции (SQL)

Таблица для демо:

```
CREATE TABLE accounts (id INT PRIMARY KEY, balance INT);
INSERT INTO accounts VALUES (1, 1000);
```

Две сессии: **T1** и **T2**. Запускать в двух окнах `psql`.

Dirty Read — чтение незафиксированных данных

Уровень: Read Uncommitted (в PostgreSQL фактически не даёт dirty read — см. М4).

```
-- T1
BEGIN;
UPDATE accounts SET balance = 500 WHERE id = 1; -- не COMMIT

-- T2 (Read Uncommitted в MySQL)
SELECT balance FROM accounts WHERE id = 1; -- может увидеть 500

-- T1
ROLLBACK; -- T2 читал «грязные» 500, которых не было
```

Non-Repeatable Read

Уровень: Read Committed допускает.

```
-- T1
BEGIN; -- PostgreSQL default: READ COMMITTED
SELECT balance FROM accounts WHERE id = 1; -- 1000

-- T2
BEGIN;
UPDATE accounts SET balance = 800 WHERE id = 1;
COMMIT;

-- T1 — повторное чтение
SELECT balance FROM accounts WHERE id = 1; -- 800 (было 1000)
COMMIT;
```

Один и тот же SELECT в транзакции T1 дал разный результат.

Phantom Read

Уровень: Repeatable Read в PostgreSQL частично закрывает, Serializable — полностью.

```
CREATE TABLE orders (id SERIAL, account_id INT, amount INT);
INSERT INTO orders (account_id, amount) VALUES (1, 100);

-- T1
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT SUM(amount) FROM orders WHERE account_id = 1; -- 100

-- T2
INSERT INTO orders (account_id, amount) VALUES (1, 200);
COMMIT;

-- T1
SELECT SUM(amount) FROM orders WHERE account_id = 1;
-- в PG Repeatable Read: snapshot — всё ещё 100
-- в Serializable / при другом движении — может быть 300 (phantom)
COMMIT;
```

Сводка

Аномалия	Read Uncommitted	Read Committed	Repeatable Read	Serializable
Dirty read	✓	✗	✗	✗
Non-repeatable read	✓	✓	✗	✗
Phantom read	✓	✓	✓*	✗

* PostgreSQL RR блокирует phantom за счёт snapshot isolation.

Go: уровень в транзакции

```
tx, err := db.BeginTx(ctx, &sql.TxOptions{
    Isolation: sql.LevelRepeatableRead,
})
```

Для перевода денег / резерва SKU на складе Magnit — минимум **Repeatable Read** или `SELECT FOR UPDATE`.

Magnit Tech: READ UNCOMMITTED в PostgreSQL

М4. Почему в PostgreSQL нет первого уровня изоляции и как это решается?

Формально уровень есть

```
BEGIN TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
```

PostgreSQL **принимает** синтаксис, но **игнорирует** его: внутри всегда работает как минимум **Read Committed**.

Проверка:

```
SHOW transaction_isolation; -- read committed (по умолчанию)
```

Почему нет настоящего READ UNCOMMITTED

PostgreSQL использует **MVCC** (Multi-Version Concurrency Control):

- Каждая транзакция видит **snapshot** зафиксированных версий строк
- Незакоммиченные изменения другой транзакции помечены «невидимыми» для читателей
- Читатель **не может** прочитать «грязную» версию строки — её просто нет в его snapshot

Dirty read противоречит модели MVCC: чтобы его разрешить, пришлось бы читать незакоммиченные tuple из чужой транзакции, ломая изоляцию и семантику отката.

Что вместо этого

Потребность	Решение в PG
Минимальные блокировки на чтение	MVCC: читатели не блокируют писателей
«Быстрое» чтение без ожидания lock	Read Committed + snapshot
Допустить грязное чтение	Не поддерживается намеренно

Read Committed в PG

- Каждый **отдельный statement** в транзакции видит snapshot на момент **начала этого statement**
- Отсюда non-repeatable read внутри одной транзакции — норма для RC

Если нужна стабильная картина

```
BEGIN ISOLATION LEVEL REPEATABLE READ;  
-- или  
BEGIN ISOLATION LEVEL SERIALIZABLE;
```

На себе

Краткий ответ: «В PostgreSQL READ UNCOMMITTED = READ COMMITTED из-за MVCC; dirty read невозможен по дизайну». Длинный: объяснить snapshot, xmin/xmax, почему это плюс для ритейла (чеки, остатки) — не читаем незафиксированные списания.

Magnit Tech: составной индекс

М5. Как выбирать порядок колонок в составном индексе?

Индекс (a, b, c) — B-tree сначала сортирует по a, внутри одинаковых a — по b, и т.д.

Правило левого префикса

Индекс (city, status, created_at) используется для:

- WHERE city = ? — да
- WHERE city = ? AND status = ? — да
- WHERE city = ? AND status = ? ORDER BY created_at — да, часто без sort
- WHERE status = ? — **нет** (пропущен city слева)
- WHERE city = ? AND created_at > ? — city да, created_at как range — дальше по индексу хуже

Порядок колонок

1. **Равенство (=, IN)** — слева, в порядке селективности (часто: более селективный первым, если оба в WHERE)
2. **Диапазон (>, <, BETWEEN)** — после equality-колонок; обычно **одна** range-колонка в индексе эффективна
3. **ORDER BY** — колонки сортировки в конце, если совпадают с направлением индекса

Пример: лента товаров магазина

```
-- Запрос
SELECT * FROM products
WHERE store_id = 42 AND category = 'dairy' AND price BETWEEN 100 AND 500
ORDER BY updated_at DESC
LIMIT 20;

-- Индекс
CREATE INDEX idx_products_store_cat_price_updated
ON products (store_id, category, price, updated_at DESC);
```

store_id, category — equality; price — range; updated_at — sort (если planner использует index scan + filter).

Covering index (INCLUDE)

```
CREATE INDEX idx_orders_user_created
ON orders (user_id, created_at DESC)
INCLUDE (total_kopecks, status);
```

Index-only scan, если все колонки SELECT в индексе.

Антипаттерны

- (created_at, store_id) при фильтре только по store_id — индекс бесполезен
- Дублировать индексы (a,b) и (a,b,c) — иногда (a,b,c) покрывает (a,b)
- Ставить low-cardinality поле первым (gender, is_active) — индекс слабый

EXPLAIN

Всегда проверять: EXPLAIN (ANALYZE, BUFFERS) SELECT ... — Index Scan vs Seq Scan, Sort отдельно или нет.

Magnit Tech: rate limiter

M6. Rate limiter на PostgreSQL и в Go на atomic (почему не int)

Задача

Лимит **N** запросов в минуту на `api_key` при нескольких репликах API — счётчик в памяти одного нод не подходит.

Вариант 1: PostgreSQL (fixed window)

```
CREATE TABLE rate_limits (  
    api_key TEXT NOT NULL,  
    window_start TIMESTAMPTZ NOT NULL,  
    count INT NOT NULL DEFAULT 0,  
    PRIMARY KEY (api_key, window_start)  
);  
  
-- В транзакции, window_start = date_trunc('minute', now())  
INSERT INTO rate_limits (api_key, window_start, count)  
VALUES ($1, $2, 1)  
ON CONFLICT (api_key, window_start)  
DO UPDATE SET count = rate_limits.count + 1  
RETURNING count;
```

Если `count > N` → 429 Too Many Requests.

Плюсы: один источник правды для всех реплик.

Минусы: нагрузка на БД на каждый запрос; для high RPS лучше Redis + TTL или sliding window в памяти gateway.

Атомарность: `INSERT ... ON CONFLICT DO UPDATE` — одна строка, row-level lock, гонки нет.

С advisory lock (альтернатива):

```
SELECT pg_advisory_xact_lock(hashtext($api_key));  
-- read count, increment, check limit
```

Держит lock до конца транзакции.

Вариант 2: Go in-process на atomic (одна реплика)

```
type Limiter struct {  
    tokens atomic.Int64  
}  
  
func (l *Limiter) Allow() bool {  
    for {  
        cur := l.tokens.Load()  
        if cur <= 0 {  
            return false  
        }  
        if l.tokens.CompareAndSwap(cur, cur-1) {  
            return true  
        }  
    }  
}
```

Пополнение: горутина с `time.Ticker` → `atomic.Store / Add`.

Почему CAS-loop: два `Load` + `Store` без CAS — lost update между горутинами.

Почему обычный int не работает

```
var tokens int  
  
func allow() bool {  
    if tokens <= 0 {  
        return false  
    }  
    tokens-- // DATA RACE: не атомарно  
    return true  
}
```

Две горутины обе читают `tokens == 1`, обе проходят проверку, обе декрементят — лимит нарушен. `go test -race` покажет гонку. Даже с `sync.Mutex` вокруг `int` — ок для одного процесса, но **не для кластера**.

Сравнение

Решение	Несколько реплик	Latency	Сложность
atomic in-memory	✗	Низкая	Низкая

Решение	Несколько реплик	Latency	Сложность
<code>int</code> без sync	✗ (и race)	—	Баг
PostgreSQL upsert	✓	Выше	Средняя
Redis INCR+EXPIRE	✓	Низкая	Средняя
API Gateway (nginx)	✓	Низкая	Инфра

На себе Magnit

Ожидают: понимание **где** state (память vs БД vs Redis), **почему** `int++` race, **как** PG обеспечивает атомарный инкремент строки, когда БД — overkill (миллионы RPS → Redis/gateway).

См. также live-coding: [go-interview-tasks/live-coding/ratelimit/](#), `atomic` в `counter/`.

Avito

Avito: скрининг — теория

A1. Go: слайс, map, set, каналы, context, panic

Функция как параметр? Да, функции — first-class values (функции высшего порядка).

Слайс vs массив: массив — фиксированный размер, value type; слайс — descriptor (ptr, len, cap) поверх массива, reference semantics.

Map lookup: в среднем $O(1)$, worst case $O(n)$ при коллизиях/ребалансе.

Set в Go: нет в stdlib; `map[T]struct{}` — нулевой value size для `struct{}`.

Каналы: буферизованные (`make(chan T, n)`) и небуферизованные (синхронная handoff).

context.Context: отмена, дедлайны, request-scoped values; прокидывать по границам API.

Exceptions: нет; для программных ошибок — `error`, для невозможного — `panic` + `recover` на границе (HTTP middleware, worker top-level).

Mutex: mutual exclusion для shared memory; `RWMutex` — много readers или один writer.

A2. БД: индексы, B-Tree, транзакции, WHERE/HAVING, триггеры

Зачем индексы: ускорить поиск/сортировку/join; примеры — B-Tree (PostgreSQL default), Hash, GiST/GIN, R-Tree.

B-Tree быстрее full scan: сбалансированное дерево, высота $O(\log n)$, страницы на диске — меньше random I/O.

Транзакция: несколько операций атомарно; **ACID** — Atomicity, Consistency, Isolation, Durability.

WHERE vs HAVING: WHERE фильтрует строки до GROUP BY; HAVING — группы после агрегации.

Агрегаты: COUNT, SUM, AVG, MIN, MAX — фильтр по результату агрегации в HAVING.

Триггер: процедура на INSERT/UPDATE/DELETE; audit, denormalization, constraints — осторожно с latency и отладкой.

A3. Сеть, HTTP, OS, контейнеры

TCP vs UDP: TCP — connection, порядок, retransmit; UDP — без гарантий, меньше overhead.

HTTP request: Request line (method, path, version) → Headers → blank line → Body.

HTTPS vs HTTP: TLS поверх TCP — шифрование, аутентификация сервера (сертификат).

HTTP timeouts: client timeout / context deadline; подбирать по p99 latency + запас, не «навсегда».

Поток vs процесс: процесс — отдельное address space; потоки одного процесса — общая память.

kill: `kill PID`, `killall name`, `pkill pattern`.

Выделение 1 KB: allocator runtime → возможно новый span от OS; при нехватке — GC, затем OOM kill.

Pod: группа контейнеров, shared network namespace и volumes в k8s.

Docker vs VM: контейнеры делят kernel хоста — легче, быстрее старт; VM — полная guest OS.

Avito: горутины и runtime

A2. Что такое горутины и как они соотносятся с потоками ОС?

Basic (ожидание на себе): - Легковесные «зелёные потоки», планируются **runtime Go**, не 1:1 с OS thread. - Модель **M:N** — много goroutines на меньшее число OS threads (GOMAXPROCS). - Переключение в user space дешевле, чем context switch потока (~KB стека goroutine vs ~MB на OS thread).

Advanced: - Goroutine блокируется на syscall/network — runtime может отцепить её от P и занять M другой goroutine. - Стек goroutine растёт динамически (от ~2 KB, copy-on-grow). - Утечки goroutine: забыли выйти из select, нет consumer у channel, WaitGroup mismatch — искать через pprof goroutine + `-race`.

Expert: - **G-M-P model:** G (goroutine), M (OS thread), P (logical processor с run queue). - Локальная очередь P + global queue + work stealing. - Sysmon будит goroutines после network poll / timer.

A2b. Чем занимается runtime?

- Планирование goroutines (scheduler).
 - Memory allocator + **GC** (tri-color concurrent mark-sweep).
 - Stack management.
 - **Netpoll** integration (epoll/kqueue/IOCP).
 - `panic / recover`, `defer`, `cgo bridge`.
-

A2c. Как Go держит много сетевых соединений?

Basic: netpoller — неблокирующие сокет; goroutine «спит», не занимая OS thread, пока нет данных.

Advanced: - Linux: **epoll** (edge/level triggered), BSD: kqueue, Windows: IOCP. - Преимущество epoll над select/poll: O(1) по числу fd, без линейного scan. - Регистрируем интересующие events: read/write/error.

Expert: edge-triggered — событие один раз до следующего `read`; level-triggered — пока буфер не пуст. Go runtime абстрагирует это в `internal/poll`.

Avito: каналы, sync, интерфейсы, память, тесты

A3. Каналы

Basic: CSP-примитив для передачи значений и синхронизации; `unbuffered` = rendezvous; `buffered` = очередь фикс. размера; `select` мультиплексирует каналы.

Advanced: - Запись в `closed` channel → panic; чтение — `zero value` + `ok=false`. - Запись/чтение в `nil` channel блокируется навсегда (иногда намеренно в `select`). - Каналы vs mutex: каналы — ownership transfer; mutex — shared state; «share memory by communicating».

A3b. Примитивы синхронизации

Примитив	Когда
Mutex / RWMutex	Инвариант на shared struct
atomic	Счётчики, flags, CAS на одной ячейке
WaitGroup	Дождаться N goroutines
Once	Однократная инициализация
Cond	Wait/signal на condition
channel	Pipeline, fan-in/out, cancellation signal

Проблемы: **data race** (`go test -race`), **deadlock** (циклическое ожидание locks/channels).

A3c. Интерфейсы

Basic: неявная реализация — тип с нужными методами; `interface{}` / `any` — любой тип.

Advanced: `iface := (type, data word)`; dynamic dispatch через `itable`.

Expert — nil interface trap:

```
var p *Person = nil
var i interface{} = p
fmt.Println(i == nil) // false - type word != nil
```

A3d. Память: slice и map

len vs cap (slice): `len` — видимые элементы; `cap` — ёмкость backing array от offset slice header.

Рост slice: при `append` beyond `cap` — новый array (~×2 до 1024, потом ~×1.25), copy, новый header.

Map: hash table; пост при load factor; iteration order random.

GC (кратко): mark reachable from roots (stacks, globals), sweep unmarked; STW только на короткие фазы; GOGC tuning.

A3e. Тесты

- Виды: unit, integration, e2e, benchmark, fuzz.
- `go test: -run, -bench, -race, -cover, -count=1, -short`.
- Mocks: интерфейс + fake / testify/mock / gomock — подменить IO boundary.

A3f. Профилирование

- **pprof:** CPU, heap, goroutine, mutex, block profiles.
- **trace:** timeline goroutines, GC, syscalls.
- CPU hot path → alloc reduction; mutex profile → contention; heap inuse → leak или oversized cache.

Включение pprof — sampling overhead, обычно приемлем в staging / short window in prod (с осторожностью).

A4. Указатели: что выведет программа?

```
type Person struct { Name string }

func changeName(person *Person) {
    person = &Person{Name: "Alice"} // меняем копию указателя
}

func main() {
    person := &Person{Name: "Bob"}
    fmt.Println(person.Name) // Bob
    changeName(person)
    fmt.Println(person.Name) // Bob
}
```

Почему: в Go нет **pass-by-reference** для параметров. `changeName` получает **копию** указателя; присваивание `person = &Person{...}` меняет только локальную копию.

Исправление — менять поле по указателю:

```
func changeName(person *Person) {
    person.Name = "Alice"
}
// или *person = Person{Name: "Alice"}
```

A4b. Горютины в цикле: максимальное чётное

```
var max int
for i := 1000; i > 0; i-- {
    go func() {
        if i%2 == 0 && i > max {
            max = i
        }
    }()
}
fmt.Printf("Maximum is %d", max)
```

Три проблемы (ожидают на собесе):

1. Loop variable capture

Все goroutines видят **одно** `i` (Go ≤1.21) или последнее значение. `go vet: loop variable i captured by func literal`.

Fix: `go func(v int) { ... }(i)`

2. Нет синхронизации завершения

`main` может выйти до старта goroutines.

Fix: `sync.WaitGroup`

3. Data race на `max`

Read-modify-write не атомапен. Atomic только на `max = i` **не спасает** — нужно лочить **и сравнение, и запись** (check-then-act).

Fix:

```
mu.Lock()
if v%2 == 0 && v > max {
    max = v
}
mu.Unlock()
```

Инструменты: `go vet`, `go test -race`, `-race` в CI.

Follow-up: зачем `defer wg.Done()` vs прямой вызов; low-cost defers (Go 1.14+); можно ли читать `max` без lock — нет, если параллельно пишут.

Avito: сборка сниппета

A5. Сборка сниппета товара

Собрать сниппет товара: **описание** (сервис → `prettify`) + **цена в рублях** (сервис в USD → `priceToRub`). Два независимых IO-вызова.

Базовое решение — параллельно через WaitGroup

```
func BuildSnippet(itemID int) (Snippet, error) {
    var wg sync.WaitGroup
    wg.Add(2)

    var desc string
    var priceRub float64
    var errDesc, errPrice error

    go func() {
        defer wg.Done()
        raw, err := itemDescription(itemID)
        if err != nil {
            errDesc = err
            return
        }
        desc = prettify(raw)
    }()

    go func() {
        defer wg.Done()
        usd, err := itemPrice(itemID)
        if err != nil {
            errPrice = err
            return
        }
        priceRub = priceToRub(usd)
    }()

    wg.Wait()
    if errDesc != nil {
        return Snippet{}, errDesc
    }
    if errPrice != nil {
        return Snippet{}, errPrice
    }
    return Snippet{Description: desc, Price: priceRub}, nil
}
```

Идиоматичнее — errgroup + context

```
func BuildSnippet(ctx context.Context, itemID int) (Snippet, error) {
    g, ctx := errgroup.WithContext(ctx)
    var desc string
    var priceRub float64

    g.Go(func() error {
        raw, err := itemDescription(ctx, itemID)
        if err != nil {
            return err
        }
        desc = prettify(raw)
        return nil
    })

    g.Go(func() error {
        usd, err := itemPrice(ctx, itemID)
        if err != nil {
            return err
        }
        priceRub = priceToRub(usd)
        return nil
    })

    if err := g.Wait(); err != nil {
        return Snippet{}, err
    }
    return Snippet{Description: desc, Price: priceRub}, nil
}
```

Первая ошибка отменяет `ctx` — второй вызов может прерваться.

Дополнительные вопросы (как на собеседе)

Пачка задач: `errgroup` или worker pool с semaphore; ограничить parallelism.

Ошибка одного сервиса: - fail whole snippet (500) — строгий контракт; - partial response (цена без описания) — явно в API contract; - fallback (USD вместо RUB) — только если продукт согласен; - retry с backoff — для transient errors, idempotent GET.

Таймауты: `context.WithTimeout` на handler; per-upstream timeout $\leq$ общего; учитывать p99 dependency + margin.

Threads vs goroutines: OS threads тяжёлые; goroutines multiplexed на GOMAXPROCS; блокирующий IO не блокирует весь процесс.

Scheduler: goroutine park на channel/network/syscall; wake через netpoll/timer.

Mutex: always defer Unlock; RWMutex если read-heavy; panic в critical section — defer всё равно отпустит lock (если не recursive misuse).

Atomic/CAS: счётчики OK; составной invariant — mutex.

Sync между машинами: не shared memory — БД, Redis lock, message queue, idempotency keys.

См. также `go-interview-tasks/live-coding/errgroup/`.

Avito: прогноз погоды 10k RPS

A6. Прогноз погоды при 10k RPS

- `aiWeatherForecast()` — ~1 сек CPU/IO (NN).
- HTTP `/weather` — **10k RPS**.
- Нельзя вызывать NN на каждый request.

Антипаттерн

```
http.HandleFunc("/weather", func(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, `{"temperature":%d}`, aiWeatherForecast()) // 1 RPS effective
})
```

Решение: кеш + фоновое обновление

```
type cache struct {
    mu      sync.RWMutex
    temperature int
}

var c cache

func refresh(ctx context.Context) {
    for {
        v := aiWeatherForecast()
        c.mu.Lock()
        c.temperature = v
        c.mu.Unlock()
        select {
        case <-ctx.Done():
            return
        case <-time.After(time.Second):
        }
    }
}

func handler(w http.ResponseWriter, r *http.Request) {
    c.mu.RLock()
    t := c.temperature
    c.mu.RUnlock()
    fmt.Fprintf(w, `{"temperature":%d}`, t)
}
```

Handler — **O(1)**, read lock; refresh — один goroutine или worker pool.

C `cityId`

- Ключ кеша: `cityId` → `map[int]entry{temperature, updatedAt}`.
- Структура: `map` + `RWMutex` или `sync.Map` для read-heavy; **не `vector`** — `cityId` sparse.
- Per-key **singleflight** или `Semaphore(1)` — thundering herd при miss не запускает 1000 NN calls.

```
import "golang.org/x/sync/singleflight"

var g singleflight.Group

func getWeather(cityID int) (int, error) {
    v, err, _ := g.Do(strconv.Itoa(cityID), func() (any, error) {
        return aiWeatherForecast(cityID)
    })
    return v.(int), err
}
```

Нештатные ситуации кеша

Проблема	Решение
Cold start	warmup top-N cities при старте
TTL expiry thundering herd	jitter TTL, singleflight, stale-while-revalidate
Stale data	TTL + background refresh до expiry
Race на map	RWMutex / sync.Map / sharded locks
High write contention	отдельный refresh goroutine, readers только читают snapshot

Таймаут на NN: context 1–2s; при timeout — last known value или 503.

HTTP: 200 OK, 400 bad cityId, 500 upstream fail — не маскировать 500 как 200.

Expert follow-ups: double buffering (swap `storage` / `oldStorage`); periodic top-N refresh; `ConcurrentDictionary` аналог в других языках.

CM. `go-interview-tasks/live-coding/singleflight/`, `ratelimit/`.

Avito: сеть, параллельные запросы, timeout

A7. IO-bound: 10 000 последовательных «сетевых» вызовов

Последовательно: ~10 s (1 ms × 10k). Параллельно:

```
var wg sync.WaitGroup
var count atomic.Int64
for i := 0; i < numRequests; i++ {
    wg.Add(1)
    go func() {
        defer wg.Done()
        networkRequest()
        count.Add(1)
    }()
}
wg.Wait()
```

Ожидание: ~1–2 ms wall time (если IO parallel). **Но:** 10k goroutines на **реальный** HTTP — risk: FD limit, upstream rate limit, memory.

Ограничение: semaphore / worker pool (например 100 concurrent).

A7b. Параллельные HTTP status codes

```
func fetchStatus(ctx context.Context, url string) error {
    req, _ := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
    resp, err := http.DefaultClient.Do(req)
    if err != nil {
        return err
    }
    defer resp.Body.Close()
    io.Copy(io.Discard, resp.Body)
    fmt.Println(url, resp.Status)
    return nil
}

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
    defer cancel()
    g, ctx := errgroup.WithContext(ctx)
    for _, u := range urls {
        u := u
        g.Go(func() error { return fetchStatus(ctx, u) })
    }
    _ = g.Wait()
}
```

Ошибки кандидатов: wg.Add внутри goroutine; sequential fetch; нет drain body (reuse connection).

Follow-ups: - 500 URLs — тот же паттерн, pool ограничивает fan-out. - Max 2s — context.WithTimeout. - Graceful shutdown — signal.Notify, server.Shutdown(ctx). - HTTP/1.1 keep-alive — Transport.MaxIdleConns, IdleConnTimeout. - Debug connections — ss, lsof -p PID. - Circuit breaker при flaky upstream.

A7c. predictableFunc — timeout обёртка

```
func predictableFunc(ctx context.Context) (int64, error) {
    ctx, cancel := context.WithTimeout(ctx, time.Second)
    defer cancel()

    start := time.Now()
    defer func() { log.Println("elapsed:", time.Since(start)) }()

    ch := make(chan int64, 1) // buffered – иначе leak goroutine при timeout
    go func() {
        ch <- unpredictableFunc()
    }()

    select {
    case v := <-ch:
        return v, nil
    case <-ctx.Done():
        return 0, ctx.Err()
    }
}
```

Критично: - **Buffered channel** — иначе goroutine зависнет после timeout. - **defer cancel()** — освободить timer. - **defer** с time.Since(start) — захват start в closure, не time.Since(time.Now()) в defer args.

select с несколькими ready channels — choice **pseudo-random**, не полагаться на порядок.

Лучше (если можно): unpredictableFunc(ctx context.Context) — native cancellation.

См. go-interview-tasks/live-coding/timeout/, shutdown/.

Avito: алгоритмические задачи

A8. Чемпионат по шагам

Условие: по дням соревнований — списки `{userId, steps}`. Найти `userId` с **максимальной суммой шагов**, участвовавших **во все дни**. Несколько победителей возможно; если никто не был все дни — пустой результат.

Сложность: $O(N)$ по всем записям или $O(N \times K)$ по дням $\times$ участников дня.

Решение

```
type stat struct {
    days    int
    steps   int
}

func getChampions(statistics [][]Statistic) Result {
    if len(statistics) == 0 {
        return Result{}
    }
    totalDays := len(statistics)
    m := make(map[int]stat, len(statistics[0]))

    for _, day := range statistics[0] {
        m[day.UserID] = stat{days: 1, steps: day.Steps}
    }
    for d := 1; d < totalDays; d++ {
        for _, rec := range statistics[d] {
            if s, ok := m[rec.UserID]; ok {
                s.days++
                s.steps += rec.Steps
                m[rec.UserID] = s
            }
        }
    }

    maxSteps := -1
    for _, s := range m {
        if s.days == totalDays && s.steps > maxSteps {
            maxSteps = s.steps
        }
    }
    if maxSteps < 0 {
        return Result{}
    }

    var ids []int
    for id, s := range m {
        if s.days == totalDays && s.steps == maxSteps {
            ids = append(ids, id)
        }
    }
    sort.Ints(ids)
    return Result{UserIDs: ids, Steps: maxSteps}
}
```

Corner cases: - несколько чемпионов с одной суммой; - нет участника все дни $\rightarrow \{\}$; - не класть в тар участников только из поздних дней (оптимизация: seed из day 0).

Anti-patterns: `includes` на каждом шаге $O(K^2)$; сортировка всего массива для ответа.

A8b. Суммарная неудовлетворённость покупателей

Условие: `goods[]`, `buyerNeeds[]`. Покупатель берёт товар с **минимальной** `|good - need|` (товаров бесконечно). Сумма разниц по всем покупателям.

Пример: `goods=[8,3,5]`, `needs=[5,6]` $\rightarrow 0 + 1 = 1$.

Варианты

Подход	Сложность
Полный перебор	$O(n \times m)$
Sort + two pointers	$O(n \log n + m \log m)$
Sort goods + binary search per need	$O(n \log n + m \log n)$

Binary search (типичное ожидание)

```
func findTotalDissatisfaction(goods, needs []uint) uint {
    sort.Slice(goods, func(i, j int) bool { return goods[i] < goods[j] })
    var total uint
    for _, need := range needs {
        total += abs(closest(goods, need), need)
    }
    return total
}

func closest(sorted []uint, target uint) uint {
    lo, hi := 0, len(sorted)-1
    for lo+1 < hi {
        mid := (lo + hi) / 2
        if sorted[mid] < target {
            lo = mid
        } else {
            hi = mid
        }
    }
    if abs(sorted[lo], target) <= abs(sorted[hi], target) {
        return sorted[lo]
    }
    return sorted[hi]
}
```

Two pointers работает если **оба** массива отсортировать и двигать указатель по goods для каждого need monotonically.

Ошибки: забыть tie-breaking при равной разнице; overflow (в задаче uint); не отсортировать goods перед бинарным поиском.